VNCKey

RUST PERÚ

Diviértete con Rust

Viaja al futuro con el mundo de Rust



🦀 Programación de Sistemas
⚡ Zero-Cost Abstractions
🔒 Memory Safety

Alexander Villanueva

Contents

1. Introducción: Cuando el ascensor se detuvo 6
    1.1. Instalación 7
        1.1.1. Linux y macOS 8
        1.1.2. Windows 8
        1.1.3. Verificación 8
    1.2. Hola Mundo 9
    1.3. Primeros Pasos en Rust 12
        1.3.1. Variables con let 12
        1.3.2. Mutabilidad con mut 12
        1.3.3. Const & static 13
        1.3.4. Shadowing 13
        1.3.5. println! 14
        1.3.6. Comentarios 16
    1.4. Bloques y Alcances (Blocks & Scopes) 17
    1.5. Declaraciones y Expresiones 18
2. Seguridad de Memoria en Rust 20
    2.1. Ownership 23
    2.2. References & Borrowing 26
    2.3. Lifetimes 29
3. Tipos de datos 30
    3.1. Tipos escalares 31
        3.1.1. Integer Types 31
        3.1.2. Floating-Point Types 33
        3.1.3. Boolean Type 35
        3.1.4. Character Type 38
    3.2. Tipos Compuestos 39
        3.2.1. Tuplas 40
        3.2.2. Array 42
    3.3. Colecciones 46
        3.3.1. String 46
        3.3.2. &str 46
        3.3.3. Vectores 46

3.4. Slices 51
3.5. Type Aliases 53
4. Control Flow 53
4.1. if Expressions 53
4.2. Bucles 56
4.2.1. Loops 56
4.2.2. While 61
4.2.3. Match 62
4.2.4. For 65
5. Iteradores 66
6. Funciones en Rust 78
7. Struct 85
7.1. Sintaxis básica 86
7.2. Instanciación de un Struct 86
7.3. Acceso a los campos 86
7.4. Structs Mutables 87
7.5. Struct Update Syntax 87
7.6. Tuple Structs 87
7.7. Unit-like Structs 88
7.8. Desestructuración de Structs 88
7.9. Derivaciones automáticas 88
7.10. Métodos en Structs 89
7.11. Self 90
7.12. Method Chaining 92
8. Enums y Patrones 94
9. Generic Types 96
10. Traits 98
10.1. Trait Bounds 102
11. Lifetimes 103
12. Manejo de Errores 108
12.1. Errores Recuperables 108
12.1.1. Option<T> 108
12.1.2. Result<T, E> 109
12.1.3. El operador ? 110

12.2. Errores NO Recuperables ... 111
12.2.1. panic! ... 111
12.2.2. Backtrace ... 112
13. Modularidad ... 113
13.1. Packages and Crates ... 113
13.1.1. Tipos de crates ... 113
13.2. Module ... 114
13.2.1. Varios archivos ... 115
13.2.2. Varios archivos en carpetas ... 116
13.2.3. Submódulos ... 116
13.2.4. Rutas (paths) ... 117
13.2.5. Use ... 117
13.2.6. super:: ... 118
13.2.7. self:: ... 120
13.2.8. crate:: ... 122
13.2.9. Control de privacidad ... 124
14. ROBUSTEZ Y ABSTRACCIONES ... 126
14.1. Más Cargo y Crates.io (publicación de paquetes). ... 126
14.2. Testing en Rust (unit, integration, benches). ... 126
15. PARADIGMAS FUNCIONALES Y OO ... 126
15.1. Operator Overloading. ... 126
15.2. Programación Orientada a Objetos en Rust ... 126
15.3. Pattern Matching Avanzado ... 126
16. SISTEMAS Y CONCURRENCIA ... 126
16.1. Smart Pointers ... 126
16.2. Threads y Concurrencia. ... 126
16.3. Async/Await y Concurrencia Ligera. ... 126
16.4. I/O Avanzado ... 126
16.5. Features Avanzados: Macros, Unsafe Rust y FFI ... 126
17. Test ... 126
17.1. Herramientas de test en Rust ... 127
17.2. Tipos de tests en Rust ... 127
17.3. Tests avanzados ... 128
18. Smart Pointers ... 129

18.1. Tipos principales de Smart Pointers 129
19. Asynchronous 132
19.1. Conceptos clave 132
19.2. Streams con API 135
20. Conclusion 136
Bibliography 136

Introducción: Cuando el ascensor se detuvo

Historia

En 2006, Graydon Hoare subía en el ascensor de su apartamento cuando el sistema se detuvo debido a un error de software. Esa frustrante experiencia lo motivó a crear un lenguaje que resolviera ese tipo de problemas, llamándolo Rust en honor al grupo de hongos del mismo nombre, organismos que sobreviven en condiciones extremas gracias a su diseño inteligente.

El problema que Hoare identificó no era nuevo, pero su solución sí lo sería.

¿Y si fuera posible unificar la velocidad y el control del lenguaje C con la seguridad y la sintaxis limpia de Python?.

Durante décadas, esa disyuntiva pareció insalvable. Rust demostró lo contrario.

¿Qué es Rust?

Rust es un lenguaje de programación de sistemas que permite crear todo tipo de aplicaciones. Integra las abstracciones modernas y la sintaxis expresiva propias de un lenguaje de alto nivel, lo que garantiza una excelente productividad con un control fino sobre el hardware que asegura la seguridad en memoria, el rendimiento y la eficiencia.

Esta convergencia da lugar a un nuevo paradigma de lenguaje, concebido para construir software confiable y eficiente, capaz de manejar concurrencia segura sin temor a errores catastróficos.

¿Qué lo hace diferente?

A diferencia de lenguajes de alto nivel como Java o Python —que incorporan un recolector de basura que introduce pausas e imprevisibilidad, Rust opera sin él. Y a diferencia de lenguajes de bajo nivel como C o C++ donde la gestión manual de memoria es poderosa pero propensa a errores catastróficos, Rust ofrece un enfoque revolucionario: reglas y conceptos verificados en tiempo de compilación que garantizan seguridad sin sacrificar rendimiento.

Gracias a su compilador estricto, Rust previene problemas como punteros nulos, fugas de memoria, accesos indebidos y en especial data races en entornos concurrentes. El resultado es un lenguaje tan cercano al hardware como C, pero con comodidades modernas y la tranquilidad de que gran parte de los errores más difíciles de depurar simplemente no pueden ocurrir.

¿Para qué sirve?

Con Rust es posible desarrollar una amplia variedad de aplicaciones: desde programas de consola, servidores web, software embebido e IoT, hasta aplicaciones de escritorio, herramientas CLI, videojuegos, e incluso sistemas operativos y controladores.

Adopción en la industria

En 2024, Rust fue nombrado el "lenguaje más querido" por octavo año consecutivo en la encuesta de Stack Overflow. Pero más allá de la popularidad, su adopción responde a hechos concretos: Amazon, Google, Microsoft y Meta lo utilizan en producción para componentes donde la seguridad y el rendimiento son innegociables.

Ejemplos de empresas que usan Rust:

Cloudflare

Reemplazó su proxy NGINX por Pingora, un marco de trabajo propio escrito en Rust. El resultado fue una reducción del 70% en el uso de CPU y del 67% en el uso de memoria para el mismo volumen de tráfico.



Microsoft

Está adoptando Rust para componentes del kernel de Windows 11. Su CTO señaló que el código en Rust es "más fácil de mantener, más fiable y más sencillo de evolucionar" en comparación con C++.



Google

Dentro de Android, la adopción de Rust ha logrado una reducción del 68% en las vulnerabilidades de seguridad de memoria, y hasta la fecha no se ha reportado ni una sola en su nuevo código de Rust.



AWS

Utiliza Firecracker, un entorno de virtualización ligero escrito en Rust, para ejecutar trillones de funciones Lambda al mes. Su código base es de solo 50k líneas, frente a las >1.4M de líneas.



Si esta introducción ha despertado tu interés, te invito a dar el siguiente paso. Nuestro cimiento será instalar Rust: un proceso sencillo de un par de minutos que nos dejará con todas las herramientas listas para escribir código que no falla en silencio.

:: Instalación

La forma oficial y recomendada de instalar Rust es a través de **rustup**, una herramienta que gestiona versiones del compilador y del ecosistema de herramientas asociadas. A diferencia de instalaciones manuales, `rustup` permite actualizar, cambiar de versión y desinstalar Rust con un solo comando.

▸ Linux y macOS

1. Abre una terminal y ejecuta el siguiente comando para descargar e iniciar el instalador:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

El flag `--tlsv1.2` garantiza que la descarga se realice sobre una conexión segura.

1. El instalador presentará tres opciones. Selecciona la opción `1` para proceder con la instalación estándar, que incluye todo lo necesario para comenzar:

```
1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>1
```

1. Al finalizar, el instalador mostrará el mensaje: *"Rust is installed now. Great!"*
2. Para que la terminal reconozca los nuevos comandos, recarga las variables de entorno sin necesidad de cerrarla:

```
source $HOME/.cargo/env
```

Alternativamente, cierra y vuelve a abrir la terminal.

▸ Windows

En Windows, Rust requiere las herramientas de compilación de C++ de Microsoft, ya que el compilador las utiliza internamente para generar ejecutables nativos.

1. Descarga e instala **Visual Studio Build Tools** desde: visualstudio.microsoft.com
2. Durante la instalación, selecciona el componente **"Desktop Development with C++"**. Si prefieres una instalación minimalista, ve a la pestaña *"Componentes individuales"* y selecciona únicamente:
   - MSVC v143 — VS 2022 C++ x64/x86 build tools
   - Windows 11 SDK (10.0.22621.0)
3. Una vez instaladas las herramientas, descarga y ejecuta el instalador oficial de Rust desde rustup.rs. Sigue el mismo proceso que en Linux y macOS.
4. Al finalizar, cierra y vuelve a abrir la terminal para que Windows reconozca los comandos `rustc`, `cargo` y `rustup`.

▸ Verificación

Independientemente del sistema operativo, ejecuta los siguientes comandos para confirmar que la instalación fue exitosa:

```
rustc --version
cargo --version
```

Una salida similar a la siguiente indica que todo está en orden:

```
rustc 1.87.0 (17067e9ac 2025-05-09)
cargo 1.87.0 (0aad5d9a4 2025-05-09)
```

> Si alguno de los comandos no es reconocido, asegúrate de haber reiniciado la terminal después de la instalación. En casos excepcionales, puede ser necesario agregar manualmente `$HOME/.cargo/bin` a la variable de entorno `PATH`.

Cargo

Es el administrador de compilación y dependencias, además de la herramienta de propósito general de Rust. Se utiliza para crear nuevos proyectos, compilar, ejecutar y administrar bibliotecas externas "dependencias".

```
cargo new mi_proyecto
```

Rustc es el compilador de Rust convierte tu código fuente en Rust a código máquina ejecutable. Aunque normalmente lo usamos a través de Cargo, rustc resulta útil para compilar archivos individuales o hacer pruebas rápidas sin crear un proyecto completo.

```
rustc mi_archivo.rs
```

Rustdoc es la herramienta de documentación de Rust. Si escribes comentarios de documentación con `///`, rustdoc puede generar automáticamente páginas HTML bien formateadas.

```
cargo doc --open
```

Rustup es el gestor del entorno Rust que permite actualizar, cambiar entre diferentes versiones del compilador "stable, beta o nightly" y administrar toolchains.

Rust se actualiza con facilidad gracias a **rustup**.

```
rustup update
```

Herramienta	Uso Rápido
cargo	cargo new / build / run
rustc	rustc archivo.rs
rustdoc	cargo doc
rustup	rustup update

Table 1: Resumen de comandos esenciales. ¡Practícalos en tu terminal!

:: Hola Mundo

Rust se integra con Cargo, su herramienta oficial para gestionar proyectos, dependencias y compilaciones.

Gracias a Cargo, crear, construir y ejecutar programas se vuelve mucho más sencillo y organizado.

Creando nuestro primer proyecto

En la terminal ejecuta:

```
cargo new mundorust
```

```
cd mundorust
```

El comando `cargo new mundorust` crea un nuevo proyecto desde cero con la estructura básica lista para programar.

```
mundorust/
├── Cargo.toml      # Archivo de configuración del proyecto
└── src/
    └── main.rs     # Archivo principal; contiene la función main()
```

Contenido de Cargo.toml

```
[package]
name = "mundorust"
version = "0.1.0"
edition = "2021"  # Edición estable actual 2021

[dependencies]
rand = "0.8"  # Ejemplo: Librería para números aleatorios
```

- `[package]`:

  Contiene los metadatos del proyecto como nombre, versión, autor y edición de Rust.

- `[dependencies]`:

  Lista las bibliotecas externas llamadas crates y Cargo las gestiona automáticamente.

El punto de entrada `main.rs`

El archivo `src/main.rs` contiene:

1. `fn main()`
   - Define la función principal del programa.
2. `{ ... }`
   - Delimita el bloque de código de la función.
3. `println!("¡Hola, mundo!")`
   - Es una macro que imprime texto en la consola.

Compilación simple sin cargo

Para pruebas rápidas, usa el compilador directo:

```
rustc main.rs
```

Esto genera un ejecutable.

- Windows:

```
.\main.exe
```

- Linux/macOS:

```
./main
```

Resultado:

```
¡Hola, mundo!
```

Compilar y ejecutar con `Cargo`

Dentro de la carpeta del proyecto, puedes compilar y ejecutar en un solo paso con:

```
cargo run
```

Salida típica:

```
   Compiling mundorust v0.1.0 (D:\VNC\RUST\mundorust)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 3.42s
     Running `target\debug\mundorust.exe`
Hello, world!
```

Si la compilación fue exitosa, ejecuta el binario generado en `target/debug/`.

Compilar sin ejecutar

Útil para verificar que el proyecto no tenga errores:

```
cargo build
```

Esto solo compila, no ejecuta.

Verificar errores rápidamente

```
cargo check
```

Ideal mientras editas el código y quieres confirmar que sigue compilando.

Ejemplo: si te olvidas un `;` o cierras mal una llave `{ }`, `cargo check` te avisará inmediatamente.

Eliminar archivos generados

```
cargo clean
```

Este comando borra los archivos creados durante las compilaciones anteriores.

Compilar para producción

Cuando necesites un binario optimizado:

```
cargo build --release
```

Esto genera una versión mucho más rápida y eficiente, ubicada en `target/release/`.

Comando	Uso principal	Cuándo usarlo
cargo new <name>	Crea un nuevo proyecto con estructura básica	Inicio de cualquier proyecto
cargo run	Compila y ejecuta en un paso	Desarrollo diario, pruebas rápidas
cargo build	Solo compila (sin ejecutar)	Verificar código sin correrlo
cargo check	Chequea errores de sintaxis/tipos (rápido)	Iteración mientras editas
cargo clean	Borra archivos generados en target/	Liberar espacio o recompilar limpio
cargo build –release	Compila optimizado para producción	Distribuir o medir rendimiento

Tabla 2: Resumen rápido de comandos Cargo. ¡Ejecuta `cargo --help` para más!

:: Primeros Pasos en Rust

En Rust, las variables son el mecanismo básico para vincular nombres a valores. Pero, a diferencia de lenguajes dinámicos como Python o JavaScript, Rust aplica una disciplina estricta desde el primer momento haciendo que las variables sean inmutables por defecto.

Esto no es una limitación, sino una característica de diseño intencional que contribuye directamente a:

- Seguridad de memoria sin recolector de basura
- Ausencia de data races en concurrencia
- Optimizaciones agresivas por parte del compilador

▸ Variables con let

La palabra clave `let` declara una variable y le asigna un valor inicial.

Rust infiere el tipo automáticamente en la mayoría de casos, pero puedes especificarlo explícitamente para claridad.

```rust
let saludo: &str = "¡Hola, Rust!";
```

```rust
saludo = "¡Hola, Rust!"; //ERROR
```

▸ Mutabilidad con mut

Si quieres que una variable pueda cambiar su valor, debes declararla con `let mut`.

Una variable mutable es aquella cuyo valor sí puede modificarse durante la ejecución del programa.

```rust
let mut y = 5; // mutable
```

```rust
let y = 5; // No es mutable
```

```
y = 20; // permitido
```

```
y = 20; // No se puede modificar
```

mut no implica que cambiarás el valor, sino que puedes hacerlo. Es una promesa al compilador para habilitar optimizaciones seguras.

▸ Const & static

Las constantes `const` y los valores estáticos `static` representan datos inmutables que viven durante toda la ejecución del programa, pero no son equivalentes.

1. `const`

   Una const es un valor:

   - evaluado en tiempo de compilación
   - inmutable
   - copiado directamente en el binario
   - nunca tiene una dirección fija de memoria

   ```
   const MAX_PUNTAJE: u32 = 100;
   ```

   Requisitos:

   - Siempre requiere tipo explícito
   - Solo puede almacenar valores que el compilador pueda evaluar como constantes
   - Usa `SCREAMING_SNAKE_CASE` por convención

     ```
     const ESPACIO: &'static str = "rust";
     ```

2. `static`

   Un valor static:

   1. vive durante toda la ejecución del programa
   2. tiene una sola ubicación fija en memoria
   3. es similar a una variable global
   4. puede ser mutable, pero requiere unsafe

   Ejemplo típico:

   ```
   static MENSAJE: &str = "Hola desde memoria estática";
   ```

   static se usa para datos globales con vida real en memoria.

▸ Shadowing

Rust permite redeclarar una variable con el mismo nombre usando **let**. Esto no modifica el valor original sino que crea una nueva vinculación que la sombrea “shadowing” dentro de su ámbito.

¿Por qué es útil?

- Permite transformar datos paso a paso, sin perder la inmutabilidad.
- Hace que el código sea más seguro y legible, ya que cada transformación es una nueva variable.
- Evita el uso de `mut` cuando solo se necesita cambiar el valor de forma temporal.

```
let x = 5;
let x = x + 1;
```

```
let x = 5;
x = x + 1;
```

Ventajas:

Permite cambiar el tipo de una variable sin perder inmutabilidad

```
let s = "   ";          // &str
let s = s.len();        // usize
```

Inferencia de Tipos y Especificación Explícita

Rust infiere tipos de forma agresiva, pero no adivina. Solo deduce a partir de expresiones concretas.

Inferencia automática:

```
let n = 42;       // i32 valor entero sin sufijo
let f = 3.14;     // f64 valor decimal
let s = "hola";   // &str
```

Anotación explícita (recomendada en interfaces públicas):

```
let edad: u8 = 24;
let temperatura: f32 = 24.5;
let mensaje: &'static str = "Listo";
```

Buenas prácticas al nombrar variables en Rust

En Rust, los nombres son sensibles a mayúsculas y minúsculas.

La convención oficial es snake_case: minúsculas con guiones bajos.

```
let numero_entero = 10;      // Bueno: snake_case

let numeroEntero = 10;       // Funciona, pero no idiomático
```

Por qué usar snake_case:

- Hace el código más legible y consistente con la comunidad Rust.
- Es la convención usada por el compilador, librerías estándar y la mayoría de proyectos profesionales.
- Ayuda a mantener un estilo uniforme en equipo.

▶ println!

println! es una macro incluida en Rust que permite imprimir texto en la consola. Aunque parece simple, tiene un sistema de formato muy poderoso y flexible.

Sintaxis básica:

```
println!("Hola mundo");
```

Pero su fuerza real está en los placeholders como {} que permiten insertar valores dentro del texto:

```
println!("La edad es: {}", 20);
```

Representan lugares donde Rust insertará valores.

Ejemplo básico:

```
let x = 10;
println!("x vale {}", x);
```

Argumentos Posicionales

Puedes usar varios {} y Rust los reemplaza en orden:

```
println!("{} + {} = {}", 3, 4, 3 + 4);
```

También puedes reutilizar argumentos usando posiciones:

```
println!("{0} ama a {1}, y {1} ama a {0}", "Juan", "Ana");
```

Argumentos Nombrados

Puedes nombrar los valores y usarlos dentro del string:

```
println!(
    "Usuario {nombre} tiene {edad} años",
    nombre = "Luis",
    edad = 25
);
```

Alineación y Relleno

Rust permite alinear texto y números dentro de un ancho fijo.

1. Alineación a la derecha

   ```
   println!("{:>6}", 42);
   ```

2. Alineación a la izquierda

   ```
   println!("{:<6}", 42);
   ```

3. Alineación centrada

   ```
   println!("{:^6}", 42);
   ```

4. Relleno con caracteres

   ```
   println!("{:*^6}", 42);
   ```

Precisión (Flotantes y Strings)

1. Para flotantes (decimales)

```
println!("{:.2}", 3.141592);
```

Ancho + precisión:

```
println!("{:>8.3}", 3.141592);
```

2. Para strings (longitud máxima)

```
println!("{:.5}", "RustLang");
```

Debug

Con primitivos:

```
println!("{:?}", 123);
println!("{:?}", true);
println!("{:?}", "Hola");
```

Tipo de identificador	Convención en Rust
Variables y funciones	snake_case
Constantes	SCREAMING_SNAKE_CASE
Tipos y estructuras	PascalCase

Tabla 3: Convenciones oficiales de nomenclatura en Rust.

▸ Comentarios

Rust soporta dos tipos principales de comentarios:

1. Comentarios de línea "`single-line`"

   Comienzan con `//` y se extienden hasta el final de la línea.

```
// Esto es un comentario
let x = 10; // También puede ir al final de una línea
```

   Útil para:

   - aclarar partes específicas del código
   - comentarios cortos
   - desactivar temporalmente líneas de código

2. Comentarios de bloque "`multi-line`"

   Comienzan con `/* y terminan con */`.

```
/*
Esto es un comentario
de múltiples líneas.
*/
```

3. Comentarios de documentación "`triple slash`"

ust usa Markdown dentro de la documentación:

```
/// Suma dos números.
///
/// # Ejemplo
/// ``
/// assert_eq!(sumar(2, 3), 5);
/// ``
fn sumar(a: i32, b: i32) -> i32 {
    a + b
}
```

Estos comentarios:

- Se procesan con cargo doc
- Generan documentación HTML profesional
- Son estándar en librerías y crates públicos

Rust también tiene documentación a nivel de módulo:

```
//! Documentación para un módulo entero
```

:: Bloques y Alcances (Blocks & Scopes)

En Rust, un bloque es una secuencia de expresiones encerradas entre llaves "{ }". Los bloques definen ámbitos léxicos "lexical scopes", y estos determinan dónde una variable es válida y cuándo se libera su memoria.

Cada bloque crea un scope "alcance", que es el espacio en el que existen las variables. Cuando el programa sale de ese bloque, las variables definidas dentro de él dejan de existir automáticamente.

Dicho de otra manera:

- Bloque : es la parte física del código `{ ... }`.
- Scope : es el concepto lógico, las reglas que dicen hasta dónde vive una variable.

Ejemplo:

```
fn main() {
    let x = 10; // "x" vive en el scope principal main

    {
        let y = 20; // "y" solo existe dentro de este bloque
        println!("Dentro del bloque: x = {}, y = {}", x, y);
    }

    // Aquí "y" ya no existe, solo "x" sigue vivo
    println!("Fuera del bloque: x = {}", x);
}
```

- La variable “x” se declara en el scope principal de main, por lo tanto existe en todo el cuerpo de la función.
- La variable “y” se declara dentro de un bloque adicional `{ ... }`. Vive solo ahí.

Cuando termina el bloque, “y” es eliminada automáticamente y ya no puede usarse.

Esto asegura que la memoria se maneje de manera segura y que cada variable tenga una vida útil clara.

Shadowing en bloques

El shadowing permite volver a declarar una variable con el mismo nombre en un scope diferente. Eso no borra la variable anterior, simplemente la “oculta” o “sombrea” mientras dure el nuevo scope.

```
fn main() {
    let numero = 5;
    println!("Scope principal: numero = {}", numero);

    {
        let numero = 20; // sombreado (shadowing)
        println!("Dentro del bloque: numero = {}", numero);
    }

    // Aquí vuelve el valor original
    println!("Fuera del bloque: numero = {}", numero);
}
```

Salida:

```
Scope principal: numero = 5
Dentro del bloque: numero = 20
Fuera del bloque: numero = 5
```

En el scope principal, numero vale 5.

Dentro del bloque, se declara otro numero que somete “shadow” al anterior y ahora vale 20.

Al salir del bloque, el numero del bloque desaparece y el original “5” vuelve a estar accesible.

Esto permite reutilizar nombres de variables en contextos diferentes sin conflictos.

:: Declaraciones y Expresiones

Rust distingue claramente entre declaraciones “**statements**” y expresiones “**expressions**”. Esta distinción es fundamental porque determina cómo se estructura el flujo de control, cómo se infiere el tipo de retorno de una función y cómo se gestionan los valores en tiempo de compilación.

En Rust, casi todo es una expresión. Solo unas pocas construcciones son declaraciones puras y esto es intencional para favorecer la expresividad y reducir efectos secundarios no deseados.

Declaraciones “Statements”

- Una declaración es una instrucción que realiza una acción, pero no produce un valor utilizable. Las declaraciones no pueden aparecer en contextos donde se espera un valor , por ejemplo, como inicializador de una variable.
- Terminan normalmente con "**;**".
- No pueden ser anidadas dentro de expresiones.
- Son imperativas: describen qué hacer, no qué valor obtener.
- Tipos comunes:

1. Declaraciones de variables : con "`let`", "`const`", etc.

```
let x = 5;          // Vincula el nombre `x` al valor `5`
```

let no es una expresión. No se puede usar en asignaciones anidadas:

```
// Error de compilación
let y = (let x = 5);  // `let` no devuelve valor, no se puede asignar
```

2. Declaraciones de expresiones descartadas.

```
x + y;              // La expresión `x + y` se evalúa, pero su resultado se
descarta
println!("Hola");   // `println!` devuelve `()`, y la llamada como declaración
descarta ese valor
```

Expresiones "Expressions"

Una expresión es cualquier construcción sintáctica que produce un valor y tiene un tipo inferible. Las expresiones no llevan punto y coma.

Características:

- No terminan en "**;**" y si lo hacen, se vuelven declaraciones.
- Se pueden anidar libremente.
- Son funcionales: describen qué valor computar.

Ejemplos comunes:

```
5 + 2               // valor: 7, tipo: `i32`
"Rust"              // valor: &str, tipo: `&'static str`
x * 3               // valor dependiente de `x`
{                   // Bloque como expresión
    let a = 2;
    let b = 3;
    a + b           // valor de retorno del bloque: 5
}
```

Seguridad de Memoria en Rust

La seguridad de memoria de Rust garantiza que [1] un programa nunca acceda a memoria inválida ni cometa errores peligrosos al manejar recursos. En lenguajes como C y C++, esta responsabilidad recae por completo en el programador, lo que abre la puerta a vulnerabilidades como desbordamientos de búfer, use-after-free, double free o data races.

Rust elimina estos problemas desde su diseño.

Mediante tres conceptos fundamentales ownership, borrowing y lifetimes, el compilador verifica en tiempo de compilación que:

- Cada dato tiene exactamente un único propietario.
- Solo puede existir una referencia mutable, o múltiples referencias inmutables, pero nunca ambas al mismo tiempo.
- Ninguna referencia puede sobrevivir al dato al que apunta.

Y todo esto se valida estáticamente, sin garbage collector y sin costo adicional en tiempo de ejecución.

Este enfoque permite escribir software de sistemas drivers, motores de renderizado, servidores altamente concurrentes o firmware con un nivel de seguridad similar al de lenguajes de alto nivel, pero manteniendo el rendimiento y el control fino característicos de C/C++.

Por ello Rust se ha convertido en la opción preferida para proyectos donde la fiabilidad es crítica: desde partes del kernel de Android y componentes del navegador Firefox, hasta sistemas embebidos, blockchain y aplicaciones WebAssembly.

Stack & Heap

Rust divide la memoria en dos grandes regiones: **stack** y **heap**.

Comprender su diferencia es esencial para entender ownership, borrowing y por qué Rust evita errores como use-after-free, double free, dangling pointers, o data races sin necesidad de un garbage collector.

1. Stack

   Funciona como una torre de platos:

   - El último que entra es el primero que sale (LIFO).
   - Añadir o quitar datos es extremadamente rápido.
   - No se busca espacio: el compilador sabe exactamente cuántos bytes ocupa cada variable.

   El stack almacena:

   - Tipos primitivos: i32, u8, bool, f64, char
   - Referencias: &T, &mut T
   - Metadatos de estructuras “dinámicas” (como String o Vec)
   - Variables locales y parámetros de funciones

```rust
fn main() {
    let x: i32 = 10;   // Se guarda en el stack porque es de tamaño fijo
    let y: f64 = 3.14; // También en el stack
}
```

```
  Stack (función main)                      Heap
----------------------------------------    -----------------
Dirección de memoria   Valor     Tipo       (vacío)
0x0010                  10       i32 (x)
0x0018                 3.14      f64 (y)
----------------------------------------    -----------------
```

Cuando termina main():

- Rust libera todo el stack automáticamente.
- No se necesita heap para este código.

Funciones y el stack

Cada vez que llamas una función, Rust crea un stack frame para ella:

- parámetros
- variables locales
- direcciones de retorno

Ejemplo:

```rust
fn suma(a: i32, b: i32) -> i32 {
    let resultado = a + b; // Todo está en el stack
    resultado
}
```

Cuando suma termina, su stack frame completo desaparece.

2. Heap

   El heap se usa para datos cuyo tamaño no es conocido en tiempo de compilación o que puede crecer dinámicamente.

   Cuando un valor necesita espacio dinámico, Rust:

   - Solicita una región libre en el heap.
   - Obtiene una dirección (puntero).
   - Guarda ese puntero (más metadatos) en el stack.

   Analogía del restaurante:

   Pides una mesa para un grupo. El anfitrión busca una libre y te da un número de mesa (puntero).

   El número se guarda contigo (stack), pero la mesa real está en el comedor (heap).

   Ejemplo con String:

```rust
fn main() {
```

```
    let s: String = String::from("Hola");
    println!("{}", s);
}
```

El String almacena:

- En el stack:
  - un puntero (ptr)
  - la longitud (len)
  - la capacidad (cap)
- En el heap:
  - los bytes reales: “Hola”

```
Stack (main)                          Heap
------------------------------------  ------------------------------
Variable   Contenido                  Dirección     Datos reales
s.ptr      0xABCD                     0xABCD        "Hola"
s.len      4
s.cap      4
------------------------------------  ------------------------------
```

¿Qué pasa al terminar main()?

Rust ejecuta automáticamente el método drop de String:

- Libera la memoria en el heap exactamente una vez.
- Elimina el stack frame completo.
- Garantiza que ninguna referencia siga apuntando a memoria inválida.

```
Stack (main - liberado)          Heap
-------------------------------  ---------------------------
s → (inaccesible)                (vacío)
-------------------------------  ---------------------------
```

Tipo	Ejemplo	Descripción
i8, i16, i32, i64, i128, isize	let x: i32 = 5;	Enteros con tamaño fijo.
u8, u16, u32, u64, u128, usize	let y: u64 = 42;	Enteros sin signo (solo positivos).
f32, f64	let pi: f64 = 3.14;	Números de punto flotante.
bool	let activo = true;	Valor lógico (true o false).
char	let letra = 'A';	Caracter Unicode (4 bytes).
Tuplas de tamaño fijo	let t = (1, 2.0, true);	Todos los elementos tienen tamaño fijo.
Arreglos de tamaño fijo	let arr = [1, 2, 3];	Longitud conocida en tiempo de compilación.
Referencias (&T, &mut T)	let r = &x;	Guardan solo una dirección de memoria.
Structs con campos fijos	struct Punto {x: i32, y: i32}	Se almacenan en stack si todos los campos son de tamaño conocido.

Tabla 4: Tipos en el Stack

Tipo	Ejemplo	Descripción
String	String::from("Hola")	Texto dinámico; contenido en el heap.
`Vec<T>`	vec![1, 2, 3]	Arreglo dinámico cuyo contenido vive en el heap.
`Box<T>`	Box::new(10)	Guarda datos explícitamente en el heap.
`HashMap<K, V>`	HashMap::new()	Mapa de clave–valor almacenado en el heap.
HashSet	HashSet::new()	Conjunto sin elementos duplicados.
`Rc<T>` / `Arc<T>`	Rc::new(String::from("x"))	Punteros con conteo de referencias; datos compartidos en heap.
`Cow<'a, T>`	Cow::from("texto")	Copia en escritura: usa stack o heap según se modifique.

Tabla 5: Tipos en el Heap

:: Ownership

Ownership en Rust es un conjunto de reglas que determinan quién es el dueño de un valor en memoria.

Estas reglas permiten que Rust garantice seguridad de memoria en tiempo de compilación, sin garbage collector.

Reglas del Ownership

- Cada valor en Rust tiene un único propietario “owner”.
- Solo puede existir un propietario a la vez.
- Cuando el propietario sale de su scope, el valor se destruye automáticamente “drop”.

Esto asegura que nunca haya dos dueños del mismo dato en el heap.

Ejemplo básico:

```
let s1 = String::from("hello");
let s2 = s1.clone();
```

Aquí, clone() duplica el contenido del heap, creando dos Strings totalmente independientes.

clone() es costoso porque copia cada byte.

```
let s1 = String::from("hello");
let s2 = s1;
```

Esto no copia el heap, solo mueve “transfiere” los metadatos del stack. Rust marca s1 como inválido después del move. Si intentas usar s1:

```
error[E0382]: borrow of moved value: `s1`
```

> Este comportamiento se llama move semantics. Rust mueve la propiedad (puntero, len, cap) sin copiar el heap, evitando costos innecesarios y garantizando seguridad.

¿Qué hace realmente let s2 = s1;?

```
Antes del movimiento              Después del movimiento
====================              ==========================

Stack                             Stack
----------------------            ---------------------------
s1.ptr → 0x1234                   (s1 inválido)
s1.len = 5                        s2.ptr → 0x1234
s1.cap = 5                        s2.len = 5
                                  s2.cap = 5
----------------------            ---------------------------

Heap                              Heap
----------------------            ---------------------------
0x1234 → "hello"                  0x1234 → "hello"
----------------------            ---------------------------
```

- No copia los datos del heap.
- Solo mueve los metadatos del stack.
- Después del movimiento, s1 queda marcado como "invalidado" para evitar dos dueños del mismo heap.

> Imagina que s1 es el dueño de una casa **"heap"**.
>
> Cuando haces `let s2 = s1;`, le entregas las llaves a s2.
>
> Desde ese momento, **"s1"** ya no puede entrar a la casa porque el compilador se lo prohíbe.

Move: Transferencia de Ownership

En Rust, la mayoría de los tipos no se copian automáticamente. Al asignarlos o pasarlos a funciones, ocurre un move:

- El ownership se transfiere.
- El valor original queda inválido.
- Se evita la doble liberación de memoria.

```rust
fn main() {
    let s1 = String::from("hola");  // "s1" es el owner inicial de "hola"

    let s2 = s1;  // Move: ownership pasa a "s2" y "s1" ya no es válido

    // println!("{}", s1);  // ¡Error en compilación! "use of moved value: `s1`"

    println!("{}", s2);  // Ok: s2 imprime "hola"

}  // s2 sale de scope: "hola" se libera automáticamente
```

Qué pasa en **move**:

- Transferencia: **"s2"** hereda el puntero, longitud y capacidad de **"s1"**.
- "s1" queda inutilizable y Rust te obliga a no usar s1, ya que previene double free.
- El heap no se copia, solo se mueve el puntero.

```
Stack (s1)          |   Stack (s1 - inválido)   |  Stack (s2)
--------------------|   --------------------    |  --------------------
ptr → 0x1234        |   (ya no accesible)       |  ptr → 0x1234
len = 5             |                           |  len = 5
cap = 5             |                           |  cap = 5
                    |                           |
Heap                |                           |  Heap
--------------------|                           |  --------------------
"hello"             |                           |  "hello"
--------------------|                           |  --------------------
```

Tipos Copy

Algunos tipos implementan el trait Copy, por lo que no se mueven, se copian:

- El valor original sigue siendo válido.
- No hay invalidación.
- No hay ownership estricto.

Ejemplo:

```
fn main() {
    let x = 5;
    let y = x;   // Se copia, no se mueve
    println!("x = {}, y = {}", x, y); // Ok
}
```

¿Por qué estos tipos se copian?

- Son pequeños.
- Tienen tamaño fijo.
- Viven completamente en el stack.

Tipos que implementan Copy

- Enteros: i32, u8, ...
- Flotantes: f32, f64
- bool
- char
- Tuplas que contengan solo tipos Copy
  - (i32, bool) es Copy
  - (i32, String) NO es Copy

Tipos que NO pueden ser Copy

Todo tipo que tenga datos en el heap:

- String
- `Vec<T>`
- `HashMap<K, V>`
- `Box<T>`
- `Rc<T>, Arc<T>`

:: References & Borrowing

En Rust, cada variable es dueña de los datos que contiene. Controla su tiempo de vida y libera la memoria al salir del scope.

Sin embargo, muchas veces queremos acceder a un valor sin apropiarnos de él.

Ahí entran las referencias.

Una referencia &T permite acceder temporalmente a un valor sin moverlo ni copiarlo.

Es un puntero seguro que no toma ownership.

```
fn main() {
    let name = String::from("Rust");

    let ref_name = &name; // referencia inmutable
```

```
    println!("El nombre es: {}", ref_name);
}
```

Aquí:

- **name** es el dueño del texto “Rust”.
- **ref_name** referencia temporal que solo observa el dato.
- Como no movimos la propiedad, todavía podemos usar “name”.

Borrowing

Crear una referencia se llama borrowing “préstamo”.

Cuando haces &x, le dices al compilador:

“Solo quiero usar este valor un momento; sigue siendo de otro.”

Rust permite dos tipos de préstamo:

- Préstamo inmutable: &T puedes leer, pero no modificar.
- Préstamo mutable: &mut T puedes leer y modificar, pero con restricciones especiales.

Préstamo inmutable vs mutable

Préstamo inmutable vs mutable

```
let mut numero = 10;

let r1 = &numero;   // referencia inmutable
```

r1 puede leer el valor, pero no modificarlo.

```
let mut numero = 10;

let r1 = &numero;   // referencia inmutable
```

r2 puede modificar el valor: ahora numero vale 15.

Reglas del Borrowing

Las reglas del borrowing son el corazón de la seguridad de memoria en Rust.

1. Puedes tener múltiples referencias inmutables &T

   ```
   let x = 10;
   let r1 = &x;
   let r2 = &x;
   ```

2. No puedes mezclar referencias mutables e inmutables activas

   ```
   let mut x = 10;

   let r1 = &x;
   ```

```
let r2 = &mut x; // ERROR
```

3. Solo puede existir una referencia mutable a la vez

```
let mut x = 10;

let r1 = &mut x;
let r2 = &mut x; // ERROR
```

Rust no permite dos referencias mutables simultáneas.

```
let mut x = 10;

{
    let r1 = &mut x;
    *r1 += 5;
} // r1 muere aquí

let r2 = &mut x;
*r2 += 10;
```

Desreferenciación y el fin de un préstamo mutable

Cuando trabajamos con referencias mutables en Rust, a veces verás el operador "*" delante de una referencia. Este operador se llama desreferenciación y sirve para acceder al valor real al que apunta una referencia.

Por ejemplo:

```
let mut x = 10;

let r1 = &mut x;  // r1 apunta a x

*r1 += 1;         // desreferenciamos r1 y modificamos a x
```

En este caso:

- r1 es una referencia mutable &mut i32
- *r1 significa "el valor al que apunta r1"
- *r1 += 1 incrementa directamente el valor de x

El operador * solo se necesita cuando quieres modificar el valor.

Finalización automática de un préstamo mutable

Considera este código:

```
fn main() {
    let mut x = 10;
```

```
    let r1 = &mut x;
    *r1 += 1;
    println!("{}", r1); // r1 deja de usarse aquí

    let r2 = &mut x;
    *r2 += 2;
    println!("{}", r2);
}
```

A primera vista, podría parecer que esto debería dar error porque:

- No puede haber dos referencias mutables activas sobre el mismo valor.
- Aun así el código compila sin problemas.

Rust detecta automáticamente cuándo dejas de usar una referencia

- r1 se usa por última vez en el println!(...)
- Después de esa línea, ya no vuelves a usar r1
- Por lo tanto, Rust considera ese préstamo mutable como terminado en ese punto
- Esto permite crear una nueva referencia mutable: r2

Resumen conceptual

- Ownership:

  "¿Quién es el dueño del valor?"
- Move:

  "Te paso el valor y dejo de tenerlo."
- Referencia (&):

  "Te presto el valor, pero sigue siendo mío."
- eferencia mutable (&mut):

  "Te presto el valor para que lo modifiques, pero nadie más puede usarlo mientras tanto."

:: Lifetimes

Un lifetime "tiempo de vida" describe qué tan larga es la vida de una referencia, es decir, durante cuánto tiempo esa referencia es válida en memoria.

Rust no tiene garbage collector. Por eso, necesita asegurarse en tiempo de compilación de que:

Ninguna referencia pueda seguir existiendo cuando el dato original ya fue destruido.

Este análisis lo hace el "borrow checker", usando lifetimes.

Ejemplo de referencia que queda apuntando a memoria destruida:

```
fn main() {
    let r;
```

```
    {
        let x = 10;
        r = &x;   // r apunta a x
    } // x se destruye aquí

    println!("{}", r); // ERROR: r apunta a un valor que ya no existe
}
```

Rust lo rechaza:

- El valor x vive solo dentro del bloque interno.
- La referencia r intenta sobrevivir más allá de x.
- El compilador detecta el desbalance de tiempos de vida y lo prohíbe.

Esto evita los famosos dangling pointers, muy comunes en C/C++.

Ejemplo correcto: la referencia vive menos que el valor original:

```
fn main() {
    let x = 10;
    let r = &x; // r apunta a x, y ambos viven en este mismo scope

    println!("{}", r); // válido
}
```

Aquí:

- x vive durante toda la función.
- r está dentro del mismo ámbito.
- La referencia nunca supera la vida del valor original.

Tipos de datos

En Rust, la forma en que representamos valores, controlamos el flujo del programa y organizamos la lógica en funciones es fundamental para construir software seguro, eficiente y escalable.

- Los tipos de datos, que determinan qué clase de información puede almacenar un programa.
- Las estructuras de control, que permiten decidir qué instrucciones se ejecutan y cuándo.
- Las funciones, que encapsulan comportamientos y permiten dividir un programa en piezas reutilizables, limpias y mantenibles.

Comprender cómo Rust maneja estos conceptos es clave, porque el compilador aplica reglas estrictas que garantizan seguridad en tiempo de compilación, evitando errores comunes como desbordamientos, uso incorrecto de tipos o accesos inválidos a memoria.

:: Tipos escalares

▸ Integer Types

En Rust, un entero es un número sin parte decimal. Pueden ser:

- con signo: tipos que comienzan con i.
- sin signo: tipos que comienzan con u.

Cada tipo tiene un tamaño específico en bits, lo que determina cuántos valores puede representar. Rust prefiere la precisión explícita, lo que evita muchos errores comunes como desbordamientos silenciosos.

Tipo	Valor mínimo	Valor máximo
u8	0	255
i8	−128	127
u16	0	65 535
i16	−32 768	32 767
u32	0	4 294 967 295
i32	−2 147 483 648	2 147 483 647
u64	0	18 446 744 073 709 551 615
i64	−9 223 372 036 854 775 808	9 223 372 036 854 775 807
u128	0	$2^{128} - 1$
i128	-2^{127}	$2^{127} - 1$
usize	0	$2^{32} - 1$ o $2^{64} - 1$
isize	-2^{31} o -2^{63}	$2^{31} - 1$ o $2^{63} - 1$

Tabla 6: Rangos numéricos de los tipos enteros primitivos en Rust.

Rust por defecto infiere los enteros como i32, ya que es rápido, común y suficiente para la mayoría de cálculos.

Ejemplo:

```
let numero = 10;     // i32 por defecto
let valor = 1000u64; // u64 mediante sufijo
```

Notación numérica

Rust permite diferentes bases numéricas:

```
let decimal = 255;       // base 10
```

```
let hexadecimal = 0xff;  // base 16
let octal = 0o377;       // base 8
let binario = 0b1111_1111; // base 2
let byte: u8 = b'A';     // byte literal (ASCII)
```

Además, puedes usar guiones bajos para mejorar la legibilidad:

```
let largo = 1_000_000;
```

Desbordamiento (Overflow)

Rust maneja el desbordamiento de dos formas:

En modo debug: Produce panic cuando se supera el rango del tipo:

```
let x: u8 = 255;
let y = x + 1; // panic: overflow
```

En modo release: Hace wrapping (ciclo numérico):

255 + 1 -> 0 255 + 2 1

Puedes usar métodos explícitos:

```
x.wrapping_add(1);   // cicla
x.checked_add(1);    // devuelve Option
x.saturating_add(1); // se queda en el máximo
x.overflowing_add(1);// devuelve (resultado, hubo overflow)
```

Conversión entre enteros

Rust no convierte tipos enteros automáticamente, debes hacerlo a mano:

```
let a: u16 = 1500;
let b: u32 = a as u32;  // conversión explícita
```

Rangos predefinidos

Todos los tipos enteros tienen constantes MIN y MAX:

```
println!("u8: {} - {}", u8::MIN, u8::MAX);
println!("i16: {} - {}", i16::MIN, i16::MAX);
```

Operaciones y métodos comunes

```
let x: i32 = 42;

println!("Abs: {}", x.abs());        // valor absoluto
println!("Pow: {}", x.pow(3));       // potencia (42^3)
println!("{}",x.is_positive());      // es positivo?
println!("{}",x.is_negative());      // es negativo?
println!("{}",x.to_string());        // convierte a una cadena de texto
```

```
println!("{}",x.signum());
```

Explicación:

- **`.abs()`** devuelve el valor absoluto.
- **`.pow(n)`** eleva el número a la potencia n.
- **`.is_positive()`** y **`.is_negative()`** devuelven true o false.
- **`.to_string()`** convierte el número a una cadena "String".

Operador	Descripción	Ejemplo	Resultado
`+`	Suma	`let numero = 15 + 5;`	`20`
`-`	Resta	`let numero = 15 - 5;`	`10`
`*`	Multiplicación	`let numero = 15 * 5;`	`75`
`/`	División	`let numero = 15 / 5;`	`3`
`%`	Módulo	`let numero = 15 % 5;`	`0`

Tabla 7: Operadores aritméticos

Operador	Equivalente a	Ejemplo
`+=`	`x = x + y`	`x += 2;`
`-=`	`x = x - y`	`x -= 3;`
`*=`	`x = x * y`	`x *= 5;`
`/=`	`x = x / y`	`x /= 2;`
`%=`	`x = x % y`	`x %= 3;`

Tabla 8: Operadores de asignación compuesta

Potencias

En Rust la función pow() es la forma estándar de elevar números:

```
let base: u32 = 2;
let cuadrado = base.pow(2); // 2² = 4
let quinta = base.pow(5);   // 2⁵ = 32
```

▸ Floating-Point Types

Los números decimales en Rust se representan mediante los tipos de punto flotante, también llamados floating-point numbers. Estos tipos permiten manejar cantidades con parte fraccionaria, por lo que se

utilizan en cálculos científicos, mediciones, operaciones matemáticas complejas y cualquier situación donde los valores no sean enteros.

Rust sigue la norma IEEE 754, el estándar internacional para números de coma flotante. Existen dos tipos principales:

Ejemplo de precisión en f32 vs f64

Tipo	Precisión	Rango
`f32`	6–9 dígitos significativos	$\pm 3.4 \times 10^{38}$
`f64`	15–17 dígitos significativos	$\pm 1.8 \times 10^{308}$

Tabla 9: Tipos de coma flotante según la norma IEEE 754

Por defecto Rust utiliza f64, ya que ofrece más precisión y, en la mayoría de procesadores modernos, su rendimiento es prácticamente igual que f32.

```
let a = 1.123456789123456;   // f64 por defecto, alta precisión
let b: f32 = 1.123456789;    // f32 explícito, pierde precisión
```

Aviso importante sobre precisión

Los números en punto flotante no pueden representar todos los decimales de forma exacta. Esto ocurre en todos los lenguajes modernos.

Ejemplo clásico:

```
println!("{}", 0.1 + 0.2);
```

Salida:

```
0.30000000000000004
```

Esto no es un error de Rust: es cómo funciona la representación binaria de los decimales en IEEE 754.

Constantes útiles y funciones de información

```
let valor: f32 = -12.26565656;

println!("Es negativo?: {}", valor.is_sign_negative());
println!("f32 min: {}, max: {}", f32::MIN, f32::MAX);
println!("f64 min: {}, max: {}", f64::MIN, f64::MAX);
println!("f64 infinito: {}", f64::INFINITY);
println!("f64 NaN: {}", f64::NAN);
```

Salida:

```
Es negativo ? : true
f32 min: -3.4028235e38, max: 3.4028235e38
f64 min: -1.7976931348623157e308, max: 1.7976931348623157e308
```

```
f64 infinito: inf
f64 NaN: NaN
```

Métodos comunes para trabajar con decimales

```
let valor: f32 = 64.0;
println!("Redondear hacia abajo (floor): {}", valor.floor());
println!("Redondear hacia arriba (ceil): {}", valor.ceil());
println!("Potencia entera: {}", valor.powi(2));
println!("Potencia decimal: {}", valor.powf(3.5));
println!("Raíz cuadrada: {}", valor.sqrt());
println!("Raíz cúbica: {}", valor.cbrt());

let valor2: f32 = -48.6945;
println!("Valor absoluto: {}", valor2.abs());
println!("Redondear al entero más cercano: {}", valor2.round());
println!("Parte entera: {}", valor2.trunc());
println!("Parte fraccionaria: {}", valor2.fract());
println!("Signo (+1 o -1): {}", valor2.signum());
println!("¿Es NaN?: {}", valor2.is_nan());
println!("¿Es infinito?: {}", valor2.is_infinite());
println!("¿Es finito?: {}", valor2.is_finite());
println!("Número con 3 decimales: {:.3}", valor2);
println!("Notación científica: {:e} , {:E}", valor2, valor2);
```

Resumen

- f32: precisión media, menor tamaño.
- f64: mayor precisión, uso por defecto.

Rust usa IEEE 754, con valores especiales como NaN, INFINITY y rangos extremos.

Los floats no representan todos los decimales exactamente.

Poseen métodos matemáticos avanzados y utilitarios muy prácticos.

▸ Boolean Type

El tipo primitivo **bool** en Rust representa valores lógicos usados para expresar condiciones, comparaciones o decisiones de control.

Solo puede tener dos valores posibles: true “verdadero” o false “falso”.

```
let es_rust_genial: bool = true;
let esta_lloviendo: bool = false;
```

Operadores Lógicos Básicos

Rust ofrece operadores lógicos clásicos para combinar o invertir valores booleanos. Aquí tienes un resumen:

Operador	Nombre	Descripción
“!”	“NOT”	“Invierte el valor lógico. Si es true, devuelve false.”
“||”	“OR”	“Devuelve true si al menos uno de los operandos es true.”
“&&”	“AND”	“Devuelve true solo si ambos operandos son true.”
“^”	“XOR”	“Devuelve true solo si uno es true, pero no ambos.”

Tabla 10: Operadores Lógicos Básicos.

Logical Not

Logical **NOT** en programación es un operador lógico que invierte expresión de un valor booleano. Si la expresión es true, el resultado de negación será false.

```
fn main() {
    let x = true;
    let y = false;

    println!("X: {}", !x);        // !(x == true)  => false

    println!("Y: {}", !y);        // !(y == false) => true

    println!("X: {}", !(2 > 5));  // '2 > 5' es false → !false = true
}
```

Logical Or

El operador “||” evalúa dos expresiones booleanas y devuelve true si al menos una de ellas es verdadera.

```
fn main() {
    let a = true;
    let b = false;

    println!("true || true = {}", a || a);    // => true

    println!("true || false = {}", a || b);   // => true

    println!("false || true = {}", b || a);   // => true

    println!("false || false = {}", b || b);  // => false

    println!("(2 < 1) || (4 > 3) = {}", (2 < 1) || (4 > 3)); // true
}
```

Logical And

El operador "**&&**" en Rust evalúa dos expresiones booleanas y devuelve **`true`** solo si ambas son **`true`**.

```
fn main() {
    let a = true;
    let b = false;

    println!("true && true = {}", a && a);   // => true

    println!("true && false = {}", a && b);  // => false

    println!("false && true = {}", b && a);  // => false

    println!("false && false = {}", b && b); // => false

    println!("(3 > 1) && (4 < 5) = {}", (3 > 1) && (4 < 5)); // true
}
```

Logical Xor

En Rust, el operador "**^**" funciona con bool como `XOR` lógico

```
fn main() {
    let a = true;
    let b = false;

    println!("true ^ true = {}", a ^ a);  // => false

    println!("true ^ false = {}", a ^ b);  // => true

    println!("false ^ true = {}", b ^ a);  // => true

    println!("false ^ flase = {}", b ^ b); // => false
}
```

Comparacion

Son expresiones que comparan dos valores y devuelven un resultado booleano: **`true`** o **`false`**.

```
fn main() {
    let a = true;
    let b = false;

    let c = 5;
    let d = 3;

    println!("true == true = {}", a == a);   // => true

    println!("true == false = {}", a == b);  // => false
```

```
    println!("false == true = {}", b == a);  // => false

    println!("false == flase = {}", b == b); // => true

    println!("5 != 3 = {}", c != d);         // => true

    println!("5 < 3 = {}", c < d);           // => false

    println!("5 > 3 = {}", c > d);           // => true

    println!("5 == 3 = {}", c == d);         // => false

    println!("5 <= 3 = {}", c <= d);         // => false

    println!("5 >= 3 = {}", c >= d);         // => true
}
```

Métodos de bool

```
fn main() {
    let v = true;

    let f = false;

    println!("v.to_string(): {}", v.to_string()); // "true"
    println!("f as u8: {}", f as u8);             // 0
    println!("v as u8: {}", v as u8);             // 1
}
```

Conversión común bool : número (true = 1, false = 0) usando `as`.

▸ Character Type

El tipo char en Rust representa un carácter Unicode, almacenado en 4 bytes “32 bits”.

Esto permite que pueda representar cualquier símbolo Unicode, no solo los caracteres ASCII tradicionales.

Así, un char puede ser:

- letras ‘A’, ‘ñ’
- números ‘7’
- símbolos ‘©’, ‘π’
- emojis ‘😎’, ‘🚀’
- caracteres de lenguajes como japonés, chino, árabe, etc.

```
let letra: char = 'R';
let numero: char = '7';
let simbolo: char = '©';
let emoji: char = '😎';
```

Los caracteres siempre van entre comillas simples (‘ ‘).

No se permite un literal con más de un carácter: ‘AB’ inválido

Caracteres ≠ Bytes

En Rust, un char no es un byte, sino un punto de código Unicode. Algunos caracteres ocupan más de un byte cuando se codifican como UTF-8, pero en un char siempre ocupan 4 bytes en memoria.

Ejemplo:

```
println!("{}", '😎' as u32); // muestra el código Unicode del emoji
```

Métodos comunes de char

Rust incluye métodos para inspeccionar propiedades del carácter:

```
let c = 'R';

println!("¿Es alfabético?   {}", c.is_alphabetic());
println!("¿Es dígito?       {}", c.is_digit(10));
println!("¿Es mayúscula?    {}", c.is_uppercase());
println!("¿Es minúscula?    {}", c.is_lowercase());
```

Salida:

```
¿Es alfabético?   true
¿Es dígito?       false
¿Es mayúscula?    true
¿Es minúscula?    false
```

Otros métodos útiles:

Salida:

```
println!("{}", 'a'.is_whitespace());   // ¿espacio/tab/enter?
println!("{}", '9'.to_digit(10));      // convierte dígito → número
println!("{}", 'ñ'.len_utf8());        // bytes en UTF-8
println!("{}", 'ñ'.len_utf16());       // unidades UTF-16
```

:: Tipos Compuestos

En Rust, los tipos compuestos son aquellos que combinan varios valores en una sola unidad de datos. A diferencia de los tipos escalares, los compuestos pueden agrupar o contener múltiples valores de uno o varios tipos.

Rust tiene dos tipos compuestos principales:

- Tuplas “tuple”
- Arreglos “array”

▸ Tuplas

En Rust, una tupla es un tipo compuesto que permite agrupar varios valores dentro de un único contenedor. Su tamaño es fijo: una vez creada, no puede crecer ni reducirse.

Una tupla es útil cuando:

- Quieres empaquetar distintos tipos de datos.
- No necesitas asignar nombre a cada campo.
- Quieres retornar varios valores desde una función.
- Deseas manipular datos agrupados de manera temporal y ligera.

Rust las define como (valor1, valor2, valor3, ...).

Características principales

- Puede contener valores de tipos distintos.

  (i32, f64, &str) es diferente de (&str, f64, i32).
- El orden importa.

  (1, “Hola”) ≠ (“Hola”, 1).
- Su tamaño es fijo.

  No se pueden agregar ni remover elementos.

Son ligeras y rápidas. Las tuplas viven en la stack cuando es posible, por lo que no requieren asignación dinámica.

Creación de tuplas

Se definen entre paréntesis:

```
let persona: (&str, i32, bool) = ("Luis", 24, true);
```

Rust también puede inferir los tipos:

```
let persona = ("Luis", 24, true);  // (&str,i32,bool)
```

Acceso por índice

Cada valor dentro de la tupla se accede usando un índice con punto **“.”**.

Los índices empiezan en 0.

```
println!("Nombre: {}", persona.0); // elemento 1 :  Luis
println!("Edad: {}", persona.1);   // elemento 2 :  24
println!("Activa: {}",persona.2);  // elemento 3 : true
```

Desestructuración

Desempaqueta la tupla en variables:

```
let persona = ("Luis", 25, true);
let (nombre, edad, activo) = persona;

println!("{}", nombre);
println!("{}", edad);
println!("{}", activo);
```

Ignorar valores con _

```
let persona = ("Luis", 25, true);
let (_, edad, _) = persona;

println!("{}", edad);
```

Debug en Tuplas

Puedes imprimir una tupla con:

```
println!("{:?}", persona); //("Luis", 24, true)
```

Siempre que todos sus elementos implementen Debug.

Mutabilidad

La tupla completa debe ser mutable para modificar uno de sus campos:

Ejemplo:

```
let mut persona = ("Luis", 25, true);
persona.0 = "Ana";
persona.1 = 30;
persona.2 = false;

println!("{:?}", persona);
```

Resultado:

```
("Rust", 26, false)
```

Tupla vacía

Rust tiene una tupla especial: la tupla vacía "`()`"

También se llama:

- unit type
- unit value

```
fn saludo() {
```

```
    println!("Hola!");
}

fn main() {
    let x = saludo();
    println!("{:?}", x);  // imprime ()
}
```

Cuando una función no retorna nada, en realidad retorna "`()`".

Rangos

```
let tupla = (1, 2, 3, 4, 5);
let (primero, .., ultimo) = tupla;
println!("primero = {}, ultimo = {}", primero, ultimo);
```

Tuplas anidadas

Puedes combinar tuplas dentro de otras:

```
let anidada = ((1, 2), (3, 4));

println!("{}", (anidada.0).1); // 2
println!("{}", (anidada.1).0); // 3
```

Tuplas con un solo elemento

Esto confunde a muchos:

```
let x = (5);    // esto NO es una tupla → solo es un i32
let y = (5,);   // esto SÍ es una tupla de un elemento
```

Las tuplas de un solo elemento deben llevar coma.

Comparaciones

Funcionan si los elementos lo permiten:

```
fn main() {
    let t1 = (1, 2);
    let t2 = (1, 3);

    println!("{}", t1 < t2); // true porque compara de izq. a der.
}
```

▸ Array

Un array en Rust es una colección de tamaño fijo de valores del mismo tipo.

Los arrays de tamaño fijo siempre se almacenan en el stack.

Esto los hace muy rápidos, pero limita su uso a tamaños pequeños o medianos.

Una vez definido, su tamaño no puede cambiar durante la ejecución del programa.

Sintaxis general

```
let nombre: [Tipo; tamaño] = [valor1, valor2, valor3, ...];
```

Creación

Se definen entre corchetes “**[ ]**”.

```
  let notas: [i32; 4] = [ 15, 18, 20, 17];
              // Indice:   0,  1,  2 , 3
```

- `let notas: [i32; 4]`, se crea un array con 4 elementos `i32`
- `[15, 18, 20, 17]`, se establece los 4 numeros separados por “**,**”.

Debug e impresión

Puedes imprimir arrays fácilmente con “**{:?}**” o “**{:#?}**” para formato legible.

```
let datos = [10, 20, 30];
println!("{:?}", datos);   // [10, 20, 30]
println!("{:#?}", datos);  // formato en líneas
```

Tipos de datos

Los arrays pueden ser de cualquier tipo que sea conocido en tiempo de compilación:

```
let cadenas = ["uno", "dos", "tres"];
let booleanos = [true, false, true];
let caracteres = ['a', 'b', 'c'];
```

Todos los elementos deben tener el mismo tipo exacto. Si mezclas tipos distintos, Rust no compila:

```
// Error:
let mixto = [1, "dos", 3.0];
```

Acceso

```
// --snip--
println!("Primera nota: {}", notas[0]); // 15
println!("Segunda nota: {}", notas[1]); // 18
println!("Tercera nota: {}", notas[2]); // 20
println!("Cuarta nota: {}",  notas[3]); // 17
```

Acceso seguro

Rust evita errores de índice fuera de rango.

```
let arr = [1, 2, 3];
println!("{}", arr[5]); // Error en tiempo de ejecución (panic)
```

Acceder fuera del rango genera un panic “error en tiempo de ejecución”.

Declaración de arrays

- Inicialización explícita

```
let valores = [5, 10, 15];  // tipo: [i32; 3]
```

- Inicialización repetida

```
let ceros = [0; 5]; // [0, 0, 0, 0, 0]
```

Esto crea un array de 5 elementos, todos con valor 0.

Mutabilidad

Para modificar un array, debe ser mutable "**mut**".

```
let mut numeros = [1, 2, 3, 4];

println!("{:?}", numeros);  // Antes [1, 2, 3, 4]

numeros[2] = 99; // modificamos el valor 3 por 99

println!("{:?}", numeros)  // Despues [1, 2, 99, 4]
```

Desestructuración

```
let numeros = [1, 2, 3, 4, 5];

// Desestructuración
let [a, b, c, d, e] = numeros;

println!("a = {}, b = {}, c = {}, d = {}, e = {}", a, b, c, d, e);
```

Desestructuración parcial

Si no te interesa todos los valores, puedes usar _ para ignorar:

```
fn main() {
    let numeros = [10, 20, 30, 40];

    let [primero, _, tercero, _] = numeros;

    println!("Primero = {}, Tercero = {}", primero, tercero);
}
```

Usando rest pattern (..)

Puedes tomar los primeros elementos y guardar el resto en otro array:

```
fn main() {
    let numeros = [100, 200, 300, 400, 500];

    let [x, y, ..] = numeros; // el resto se ignora
```

```
    println!("x = {}, y = {}", x, y);

    let [a, b, rest @ ..] = numeros; // guardamos el resto en `rest`
    println!("a = {}, b = {}, resto = {:?}", a, b, rest);
}
```

Resultado:

```
x = 100, y = 200
a = 100, b = 200, resto = [300, 400, 500]
```

Comparaciones

Puedes comparar arrays directamente si sus elementos implementan PartialEq o Ord.

```
let a = [1, 2, 3];
let b = [1, 2, 3];
let c = [1, 2, 4];

println!("{}", a == b); // true
println!("{}", a < c);  // true (comparación lexicográfica)
```

La comparación es elemento por elemento, de izquierda a derecha.

Copia y movimiento

Los arrays se copian completamente si contienen tipos que implementan el rasgo Copy como enteros, booleanos o caracteres.

```
let a = [1, 2, 3];
let b = a; // copia completa del array
println!("{:?}", a); // válido, no se mueve
```

Esto ocurre porque los arrays de tamaño fijo pequeño están en el stack y son baratos de copiar.

Arrays multidimensionales

```
fn main() {
    let matriz: [[i32; 3]; 2] = [
        [1, 2, 3],
        [4, 5, 6],
    ];

    println!("Elemento fila 1, col 2: {}", matriz[0][1]); // 2
}
```

Métodos útiles

```
fn main() {
    let arr = [5, 10, 15, 20];
```

```
    println!("Longitud: {}", arr.len());
    println!("Está vacío? {}", arr.is_empty());
    println!("Primer elemento: {:?}", arr.first());
    println!("Último elemento: {:?}", arr.last());
}
```

Resultado:

```
Longitud: 4
Está vacío? false
Primer elemento: Some(5)
Último elemento: Some(20)
```

Convertir una Tupla en Array

```
let tuple: (u32, u32, u32) = (1, 2, 3);
let array: [u32; 3] = tuple.into();
```

:: Colecciones

▸ String

▸ &str

▸ Vectores

En Rust, un vector **`Vec<T>`** es una colección dinámica de elementos del mismo tipo. Y vive en el heap.

Es la colección más usada en Rust porque combina flexibilidad y seguridad.

A diferencia del array `[T; N]`, el vector tiene tamaño dinámico: puede crecer o reducirse en tiempo de ejecución.

Creación de vectores

1. Usando la macro `vec![]`

   ```
   let v = vec![1, 2, 3, 4];
   ```

2. Creación explícita con `Vec::new()`

   ```
   let mut v: Vec<i32> = Vec::new();
   ```

3. Vector repetido

   ```
   let v = vec![0; 5]; // [0, 0, 0, 0, 0]
   ```

4. Vector apartir de otro vector

   ```
   let v1 = vec![1, 2, 3, 4, 5];
   ```

```
let v2 = v1.clone(); // crea una copia exacta
```

5. Vector a partir de un iterador

```
let v: Vec<i32> = (0..5).collect();

println!("{:?}", v)
```

6. Vector con cierta capacidad

```
let mut v = Vec::with_capacity(2);
```

Rust reservará espacio en memoria desde el comienzo.

Acceso

Cuando accedes a un índice en un Vector, puedes hacerlo de dos maneras.

1. Usando índice directo

```
let v = vec![10, 20, 30];
          //  0,  1,  2
println!("Primer elemento: {}", v[0]);
```

   Si accedes a un índice fuera de rango, el programa entra en `panic` en tiempo de ejecución.

2. Usando **get**

   Con `.get()` es más seguro

```
let v = vec![10, 20, 30];
println!("{:?}", v.get(1)); // Some(20)
println!("{:?}", v.get(5)); // None
```

Mutabilidad

Los vectores pueden modificarse si se declaran **mut**:

```
let mut v = vec![10, 20, 30];

v[1] = 50; // 20 => 50

println!("{:?}", v); // [10, 50, 30]
```

Agregar elementos

```
let mut v = vec![1, 2];

v.push(3);

v.push(4);

println!("{:?}", v); // [1,2,3,4]
```

Mueve todos los elementos dejando vacío.

```
let mut vec = vec![1, 2, 3];
let mut vec2 = vec![4, 5, 6];
vec.append(&mut vec2);

println!("{vec:?}");  // [1, 2, 3, 4, 5, 6]
println!("{vec2:?}"); //[]
```

```
let mut vec = vec!["hello"];
vec.resize(3, "world");

println!("{vec:?}");
```

Insertar elementos

```
let mut v =     vec![ 10, 20, 30, 40, 50];
                  // 0   1   2   3   4
//insert(2,22)            20,|22|,30,...

v.insert(2,22);

println!("{:?}", v); // [10, 20, 22, 30, 40, 50]
```

Eliminar elementos

1. Eliminar por índice : **remove**

```
let mut v = vec![10, 20, 30, 40];

v.remove(1); // elimina el elemento en índice 1 => 20

println!("{:?}", v); // [10, 30, 40]
```

2. Eliminar el último elemento : **pop**

```
let mut v = vec![1, 2, 3];

v.pop();

println!("{:?}", v); // [1,2]
```

3. Eliminar un rango de elementos : **drain**

```
let mut v = vec![ 10, 20, 30, 40, 50];
             //  0,  1,  2,  3,  4
    //[1..3]         |.......|     => (1,2) = [20,30]

v.drain(1..3);
```

```
println!("{:?}", v); // [10, 40, 50]
```

4. Vaciar todo el vector : **clear**

```
let mut v = vec![10, 20, 30];

v.clear(); // elimina todo

println!("{:?}", v); // []
```

5. Acorta el vector, manteniendo los primeros elementos y eliminando el resto.

```
let mut v = vec![1, 2, 3, 4, 5];

v.truncate(2);

println!("{:?}", v); // [1,2]
```

Eliminar todo:

```
let mut vec = vec![1, 2, 3];

vec.truncate(0);

println!("{:?}", vec) // []
```

6. El elemento eliminado se reemplaza por el último elemento del vector.

```
let mut v = vec!["foo", "bar", "baz", "qux"];

let data = v.swap_remove(1);

println!("{:?}", data);
println!("{:?}", v);
```

Intercambio

```
let mut v = ["a", "b", "c", "d", "e"];
v.swap(2, 4);
```

Reverse

```
let mut v = [1, 2, 3];
v.reverse();
```

Longitud y capacidad

Rust distingue entre:

- **len()** cantidad actual de elementos.

- **`capacity()`** espacio reservado en memoria.

```
let mut v = Vec::with_capacity(10);
println!("len: {}, cap: {}", v.len(), v.capacity());
v.push(1);
println!("len: {}, cap: {}", v.len(), v.capacity());
```

Cuando la capacidad se llena, el vector duplica automáticamente su espacio en memoria.

Conversión de arrays a vectores

Puedes convertir fácilmente un array en vector:

```
let arr = [1, 2, 3];
let v = arr.to_vec();
```

O también con el método **`Vec::from()`**:

```
let v = Vec::from([1, 2, 3]);
```

Copia y movimiento

Los vectores no implementan **Copy**, solo **Clone**.

Al asignarlos, se mueven no se copian automáticamente.

```
let v1 = vec![1, 2, 3];
let v2 = v1; // v1 se mueve
// println!("{:?}", v1); Error: valor movido
```

Para mantener ambos:

```
let v1 = vec![1, 2, 3];
let v2 = v1.clone();
println!("{:?}", v1);
println!("{:?}", v2);
```

Comparaciones

Puedes comparar vectores si sus elementos implementan **PartialEq** o **Ord**:

```
let a = vec![1, 2, 3];

let b = vec![1, 2, 3];

let c = vec![1, 3, 2];

println!("{}", a == b); // true

println!("{}", a < c);  // true
```

Vectores multidimensionales

Puedes crear vectores dentro de otros vectores, es decir, matrices dinámicas:

```
let matriz = vec![
    vec![1, 2, 3],
    vec![4, 5, 6],
    vec![7, 8, 9],
];
println!("{}", matriz[1][2]); // 6
```

Métodos comunes

```
let mut v = vec![3, 1, 4];
v.sort(); // Ordena los elementos
println!("{:?}", v); // [1, 3, 4]
println!("{:?}", v.contains(&4));
println!("{:?}", v.first());
println!("{:?}", v.last());
v.reverse();  // Invierte el orden de los elementos
println!("{:?}", v);
println!("{:?}", v.starts_with(&[3]));
println!("{:?}", v.ends_with(&[3]));
println!("{:?}", v.repeat(3));

["hello", "world"].concat(), "helloworld")
["hello", "world"].join(" "), "hello world")
["hello", "world"].connect(" "), "hello world")

let bits = vec!["veni", "vidi", "vici"];
println!("Estado: {:?}",bits.concat());
println!("Estado: {:?}",bits.join(" - "));
println!("¿Vacío? {}", v.is_empty());
println!("Extender {}", v.extend([5,6,9]));
```

:: Slices

Un slice es una vista o referencia a una porción de una colección (array, vector, string).

- No es dueño de los datos, solo apunta a ellos.
- Guarda una referencia y una longitud.
- Muy útil para trabajar con subconjuntos sin copiar datos.

Tipo: `&[T]` (slice inmutable) o `&mut [T]` (mutable).

Crear slices

```
let vec = vec![10, 20, 30, 40, 50];
```

```rust
let slice = &vec[1..4];

println!("{:?}",slice); // [20, 30, 40]

let slice2 = &arr[..2];

println!("{:?}",slice2); // elementos 0,1 → [10,20]

let slice3 = &arr[3..];

println!("{:?}",slice3); // elementos 3,4 → [40,50]

let slice4 = &arr[..];

println!("{:?}",slice4); // todos los elementos
```

Slice mutable

```rust
let mut arr = [1, 2, 3, 4];

let slice = &mut arr[1..3]; // [2, 3]

slice[0] = 99;  // 2 => 99

println!("{:?}", arr); // [1, 99, 3, 4]
```

Slices en Strings

Un `&str` es en realidad un slice de bytes UTF-8 de un String.

```rust
let texto = String::from("Hola Rust");

let slice = &texto[0..4]; // "Hola"

println!("{}", slice);
```

Propiedades de un slice

- Longitud : `.len()`
- Vacío : `.is_empty()`
- Primer elemento : `.first()`
- Ultimo elemento : `.last()`
- Si dentro de sus elementos contiene : `.contains(&v)`

En resumen:

- Un slice `&[T]` es una referencia a parte de un array o vector.

- Es dinámico en longitud, pero no es dueño de los datos.
- Muy eficiente porque evita copiar.

:: Type Aliases

En Rust, la palabra clave type sirve para crear alias de tipos. Un alias no es un tipo nuevo, sino un nombre alternativo para un tipo ya existente, que hace que tu código sea más legible o expresivo.

Ejemplo:

```
type Meters = f32;

fn main() {
    let height: Meters = 1.74;
    let width: Meters = 0.50;
    let area: Meters = height * width;

    println!("El area es: {} metros cuadrados", area);
}
```

Aquí, Meters no es un tipo distinto de f32, sino un nombre alternativo.

Lo usamos para que sea más claro que nuestras variables representan metros, no cualquier número decimal.

Control Flow

Introducción al flujo de control

El flujo de control es la forma en que un programa decide qué instrucciones ejecutar y en qué orden. Gracias a él, un programa puede tomar decisiones, repetir tareas o seleccionar diferentes caminos según las condiciones que se presenten durante la ejecución. Sin control de flujo, un programa simplemente ejecutaría las instrucciones de arriba hacia abajo sin ninguna lógica adaptable.

Rust ofrece varias construcciones de control como: (if, loop, while, for y match) que permiten crear comportamientos dinámicos y seguros.

:: if Expressions

Las if expressions permiten tomar decisiones basadas en una condición lógica.

En Rust, if no es solo una estructura de control es una expresión, lo que significa que puede retornar un valor.

Sintaxis de un if expression:

```
if condicion {
```

```
    // rama verdadera
} else {
    // rama falsa
}
```

Ejemplo:

```
fn main() {
    let numero = 7;

    if numero > 5 {
        println!("El número es mayor que 5");
    } else {
        println!("El número es menor o igual a 5");
    }
}
```

if como expresión

Como if devuelve un valor, puedes asignarlo directamente a una variable:

```
let mensaje = if numero > 5 {
    "El numero 10 es mayor que 5"
} else {
    "El numero 10 es menor o igual a 5"
};

println!("{}", mensaje);  //El numero 10 es mayor que 5
```

If expressions anidados También puedes encadenar múltiples condiciones:

```
let numero = 15;

let resultado:&str = if numero < 0 {

    "Negativo"

} else if numero == 0 {

    "Cero"

} else {
    "Positivo"
};
println!("{}", resultado); //Positivo
```

Tipos de retorno en if expressions

Regla importante en Rust:

Todas las ramas de un if deben devolver el mismo tipo.

```
let x:i32 = 5;
let y:i32 = if x > 0 { 1 } else { -1 }; // ambas ramas son i32
```

Esto seria un error:

```
let x = 5;
let y = if x > 0 { 1 } else { "menor o igual a cero" };
// Error: tipos distintos (i32 vs &str)
```

If con bloques de código

Las ramas pueden contener bloques completos.

El valor retornado es la última expresión del bloque:

```
let x = 2;

let res = if x % 2 == 0 {
    println!("Es par");

    "par"     // valor devuelto
} else {
    println!("Es impar");

    "impar"
};

println!("Resultado: {}", res);
```

if con shadowing

```
let edad = 20;

let edad = if edad >= 18 {
    "adulto"
} else {
    "menor"
};

println!("{}", edad);
```

if dentro de expresiones más grandes

if es 100% una expresión, no solo un control de flujo.

```
let mensaje = format!(
    "Estado: {}",
    if x > 0 { "positivo" } else { "no positivo" }
```

```
);

println!("{}", mensaje);
```

Detalles importantes

- No se usan paréntesis alrededor de la condición.
  - Correcto: if x > 0
  - Incorrecto: if (x > 0)
- Siempre se requiere { } para las ramas.

Importante en Rust:

- No necesitas paréntesis `()` alrededor de la condición.
  - Correcto : `if x > 0`
  - Incorrecto : `if (x > 0)`.
  - Los bloques deben estar entre `{ }`.

Operadores lógicos

Rust soporta los operadores lógicos más comunes:

- && : AND lógico.
- || : OR lógico.
- ! : NOT negación.

```
let x = false;
let y = true;

if x && (2 > 1) {
  println!("No se ejecuta");
}

if y || (2 < 1) {
  println!("Se ejecuta sin evaluar 2 < 1");
}

if !x {
  println!("!x = true");
}
```

:: Bucles

▶ Loops

En Rust, loop ejecuta un bloque de código indefinidamente hasta que se detenga manualmente con break. Es el bucle más básico y flexible del lenguaje.

Ejemplo:

loop repite el código dentro de `{}` de manera infinita hasta que ocurra un break.

```
let mut contador = 0;

loop {
    contador += 1;
    println!("Contador: {}", contador);

    if contador == 5 {
        break; // salimos del loop
    }
}
```

Explicación del codigo:

1. `loop {`
   - Iniciamos un bucle infinito con `loop`.
   - Este bloque se repetirá hasta que lo detengamos con un `break`.
2. `contador += 1;`
   - Se incrementa el valor de contador en `1` en cada iteración.
3. `println!("Contador: {}", contador);`
   - Imprime el valor actual de `contador` en la consola.
4. `if contador == 5 { break; }`
   - Cuando contador llega a 5, el bucle termina.

Resultado:

```
Contador: 1
Contador: 2
Contador: 3
Contador: 4
Contador: 5
Fin del loop
```

loop con continue

continue permite saltar inmediatamente a la siguiente iteración del bucle.

```
let mut n = 0;

loop {
```

```
    n += 1;

    if n % 2 == 0 {
        continue; // saltamos los pares
    }

    println!("Número impar: {}", n);

    if n > 7 {
        break;
    }
}
```

1. `if n % 2 == 0 {continue;}`

- Evalua si ell número es par.
- continue hace que se ignore el resto del bloque y pase a la siguiente vuelta.
- Por eso solo se imprimen los números impares.

Salida:

```
Número impar: 1
Número impar: 3
Número impar: 5
Número impar: 7
Número impar: 9
```

loop como expresión

En Rust, casi todo es una expresión, incluyendo loop.

Puedes usar break para devolver un valor.

```
fn main() {
    let mut contador = 0;

    let resultado = loop {
        contador += 1;

        if contador == 10 {
            break contador * 2; // devuelve 20
        }
    };

    println!("El resultado es {}", resultado);
}
```

Etiquetas en bucles

Las etiquetas permiten controlar loops anidados indicando exactamente de cuál quieres salir o continuar.

```
fn main() {
    let numeros = vec![10, 20, 30, 40, 50];
    let mut indice = 0;

    'busqueda: loop {
        if indice >= numeros.len() {
            println!("Número no encontrado.");
            break 'busqueda;
        }

        let valor = numeros[indice];

        if valor == 30 {
            println!("¡Encontramos el número {} en la posición {}!", valor,
            indice);
            break 'busqueda;
        }

        indice += 1;
    }

    println!("Fin del programa.");
}
```

Explicación

- La etiqueta `'busqueda`: identifica este loop.
- `break 'busqueda` permite salir específicamente de este bucle muy útil si hay loops dentro de loops.
- El programa busca el número 30 dentro del vector y rompe el bucle cuando lo encuentra.

Salida:

```
¡Encontramos el número 30 en la posición 2!
Fin del programa.
```

Ejemplo de loops anidados:

Este ejemplo muestra por qué existen las etiquetas: para poder romper bucles externos desde dentro de bucles internos.

```
fn main() {
    let mut fila = 0;
    let mut columna = 0;
```

```
    'outer: loop {       // loop externo
        println!("Fila {}", fila);

        'inner: loop {   // loop interno
            println!("  Columna {}", columna);

            if columna == 2 {
                break 'outer; // rompemos el loop externo
            }

            columna += 1;
        }

        fila += 1;
    }

    println!("Fin del programa.");
}
```

Explicación:

- ‘outer es el loop externo.
- ‘inner es el loop interno.
- En lugar de romper solo el loop interno, usamos break ‘outer para salir directamente del externo.

Resultado esperado:

```
Fila 0
  Columna 0
  Columna 1
  Columna 2
Fin del programa.
```

Ejemplo de continue ‘label

Aunque se usa menos que break ‘label, también puedes saltar a la siguiente iteración del loop etiquetado.

```
fn main() {
    let mut fila = 0;
    let mut columna;

    'outer: loop {
        if fila > 2 {
            break;
        }
```

```
        columna = 0;

        'inner: loop {
            if columna > 3 { break; }

            if columna != 2 {   // solo imprimimos si no es 2
                println!("Fila {}, Columna {}", fila, columna);
            }

            columna += 1;       // siempre avanzamos
        }

        fila += 1;
    }
}
```

▸ While

El bucle while repite un bloque de código mientras una condición booleana sea verdadera. Es útil cuando no sabes de antemano cuántas iteraciones habrá, pero sí tienes una condición que determina cuándo detenerte.

Sintaxis:

```
while condicion {
    // código que se ejecuta mientras condicion sea true
}
```

Ejemplo basico:

```
fn main() {
    let mut contador = 0;

    while contador < 5 {
        println!("contador = {}", contador);
        contador += 1; // importante: actualizar la condición en algún punto
    }
}
```

Explicación:

- Antes de cada iteración se evalúa contador < 5.
- Si es true, se ejecuta el bloque; si es false, se sale del while.
- Es responsabilidad del programador asegurarse de que la condición cambie (normalmente modificando variables dentro del bucle); de lo contrario el bucle será infinito.

Resultado:

```
contador = 0
contador = 1
contador = 2
contador = 3
contador = 4
```

while con break y continue

Dentro de un while puedes usar break para salir inmediatamente o continue para saltar al siguiente ciclo.

```
fn main() {
    let mut n = 0;

    while n < 10 {
        n += 1;

        if n % 2 == 0 {
            continue; // saltar el resto del bloque para los pares
        }

        println!("Número impar: {}", n);

        if n >= 7 {
            break; // salir del while cuando n >= 7
        }
    }
}
```

Explicación:

- continue salta a la evaluación de la condición (n < 10) sin ejecutar las líneas siguientes dentro del bloque.
- break termina el while inmediatamente.

▸ Match

El match es una de las herramientas más poderosas del flujo de control en Rust. Permite tomar decisiones basadas en patrones, de una manera clara, segura y completamente exhaustiva. A diferencia de un simple if, match compara un valor contra múltiples patrones y obliga al programador a manejar todos los casos posibles.

Sintaxis:

```
match valor {

    Patrón1 => { /* código */ }
```

```
    Patrón2 => { /* código */ }

    _ => { /* código por defecto */ }

}
```

- valor : expresión a evaluar.
- Cada brazo (=>) representa un caso.
- El patrón _ es el caso “cualquiera”; actúa como un else.

Ejemplo basico:

La forma más simple de match consiste en comparar un valor con varias alternativas:

```
fn main() {
    let numero = 3;

    match numero {
        1 => println!("Uno"),
        2 => println!("Dos"),
        3 => println!("Tres"),
        _ => println!("Otro número"),
    }
}
```

Resultado:

```
Tres
```

Patrones

Los patrones permiten describir de forma declarativa qué valores queremos capturar o comparar. Rust soporta patrones muy expresivos:

Patrón literal

```
match x {
    0 => println!("Cero"),
    5 => println!("Cinco"),
    _ => println!("Otro valor"),
}
```

Patrón con múltiples alternativas

```
match x {
    1 | 2 | 3 => println!("Uno, dos o tres"),
    _ => println!("Otro"),
}
```

Patrones con variables

```
match x {
    10 => println!("Diez"),
    otro => println!("Otro valor: {}", otro),
}
```

Rangos

Rust permite usar rangos como patrones. Esto funciona tanto para enteros como para caracteres:

```
let edad = 20;

match edad {
    0..=12 => println!("Niño"),
    13..=17 => println!("Adolescente"),
    18..=59 => println!("Adulto"),
    _ => println!("Mayor de edad"),
}
```

Resultado:

```
Adulto
```

Destructuración simple

match también puede abrir o "desestructurar" datos, como tuplas o enums simples.

```
fn main() {
    let punto = (0, 5);

    match punto {
        (0, y) => println!("Está en el eje Y, valor: {}", y),
        (x, 0) => println!("Está en el eje X, valor: {}", x),
        (x, y) => println!("En coordenadas ({}, {})", x, y),
    }
}
```

Resultado:

```
Está en el eje Y, valor: 5
```

Condiciones lógicas

```
fn main() {
    let numero:i8 = 8;
    match numero {
        value if value % 2 == 0 => println!("El numero {numero} es par"),
        value if value % 2 == 1 => println!("El numero {numero} es impar"),
         _ => unreachable!(), // aquí decimos que nunca debería entrar
```

```
    }
}
```

Aquí, el patrón `value if ...` nos permite agregar condiciones adicionales dentro del match.

match como expresión

En Rust, match devuelve un valor, igual que un if expresión. Esto permite asignar el resultado:

```
let numero = 2;

let texto = match numero {
    1 => "uno",

    2 => "dos",

    3 => "tres",

    _ => "desconocido",
};

println!("Número: {}", texto);
```

Esto vuelve a match más poderoso que en muchos otros lenguajes, pues no solo ejecuta código: produce valores.

▸ For

El bucle for es la forma más idiomática y segura de iterar en Rust.

Rust no usa índices manuales como en otros lenguajes, sino que itera directamente sobre iteradores, lo que evita errores como desbordes, índices fuera de rango o ciclos infinitos.

for sobre rangos

La forma más común de usar un for es con rangos (start..end):

```
for i in 0..5 {
    println!("i = {}", i);
}
```

Salida:

```
i = 0
i = 1
i = 2
i = 3
i = 4
```

Notas importantes sobre rangos:

- a..b , incluye a pero excluye b.

- Para incluir el final, usa a..=b:

```
for n in 1..=3 {
    println!("{}", n);
}
```

Salida:

```
1
2
3
```

Los rangos se convierten automáticamente en iteradores, por eso funcionan con for.

Iteradores

El bucle for es uno de los más utilizados en Rust para iterar de manera segura y eficiente sobre rangos de valores o colecciones de datos. A diferencia de lenguajes como C, Rust no usa for con índices manuales esto para evitar errores comunes como off-by-one, sino que se basa en el trait IntoIterator. Esto hace que el código sea más idiomático, genérico y libre de errores de memoria.

Sintaxis básica:

```
for item in iterable {
    // Código que usa `item` en cada iteración
}
```

- **`iterable`**: Cualquier cosa que se pueda convertir en un iterador (rangos, vectores, arrays, etc.).
- **`item`**: Variable que recibe cada elemento por valor, referencia o desestructurada.
- Internamente, el **for** llama a **`into_iter()`** para crear un iterador y usa **`next()`** en un bucle while let Some(...) hasta que devuelve None.

El **for** toma ownership de la colección por defecto, pero puedes iterar por referencia **“`&colección`”** o mutable **“`&mut colección`”** para preservarla. Veremos esto en detalle más adelante.

Rangos

Los rangos son iteradores lazy “no crean arrays en memoria” y son perfectos para bucles numéricos. Se definen con **“`..`”** exclusivo o **“`..=`”** inclusivo.

- `0..5`: **Itera** de 0 a 4.
- `0..=5`: **Itera** de 0 a 5.

Ejemplo básico:

```
fn main() {
    for i in 0..5 {
        println!("{}", i);  // Salida: 0 1 2 3 4
    }
```

```
    for i in 0..=5 {
        println!("{}", i);  // Salida: 0 1 2 3 4 5
    }
}
```

- Reverso: Usa "**`.rev()`**" para iterar descendente.

```
fn main() {
    for i in (0..=5).rev() {
        println!("{}", i);  // Salida: 5 4 3 2 1 0
    }
}
```

- Paso personalizado: "**`.step_by(n)`**" salta elementos.

```
fn main() {

    for i in (0..10).step_by(2) {

        println!("{}", i);  // Salida: 0 2 4 6 8
    }
}
```

> Los rangos implementan **IntoIterator** automáticamente. Son eficientes (O(1) por elemento) y seguros (panic en debug si desbordan en modo checked).

Iterando sobre Colecciones: Arrays y Vectores

Las colecciones como arrays "**`[T; N]`**", tamaño fijo y vectores "**`Vec<T>`**" dinámico, son iterables vía métodos como `into_iter()`, `iter()` y `iter_mut()`.

- **`into_iter()`**: Toma ownership y consume la colección (mueve elementos). Útil cuando quieres destruirla intencionalmente.
- **`iter()`**: Itera por referencias inmutables (&T). Preserva la colección.
- **`iter_mut()`**: Itera por referencias mutables (&mut T). Permite modificaciones sin consumir.

Iterando Arrays en Rust

En Rust, los bucles for permiten recorrer los elementos de un array de distintas maneras, dependiendo de cómo se accede a cada elemento:

- Por valor: se obtiene una copia del elemento.
- Por referencia: se accede sin mover ni copiar.
- Por referencia mutable: se accede para modificar los elementos.

Iteración por valor

Esta forma recorre los valores del array directamente.

```
fn main() {
    let arr1 = [100, 200, 300];

    println!("Iterando con for in (por valor):");
    for x in arr1 {
        println!("{}", x);
    }

    // Los arrays de tipos Copy (como i32) siguen disponibles
    println!("Array original: {:?}", arr1);
}
```

> En arrays de tipos que implementan Copy, el array no se mueve. Pero en colecciones no Copy como "**`Vec<String>`**", se perdería la propiedad.

Iteración con .into_iter()

Convierte el array en un iterador que consume sus elementos.

```
fn main() {
    let arr2 = [400, 500, 600];

    println!("Iterando con into_iter():");
    for x in arr2.into_iter() {
        println!("{}", x);
    }

    // En arrays de Copy (como i32) sigue disponible,
    // pero en vectores o structs NO.
    println!("Array original aún usable: {:?}", arr2);
}
```

> "**`into_iter()`**" transfiere la propiedad de cada elemento al bucle.
>
> Úsalo cuando ya no necesites el array después.

Iteración con .iter() por referencia

Se usa para prestar los valores sin moverlos.

```
fn main() {
    let arr3 = [700, 800, 900];

    println!("Iterando con iter():");
    for &x in arr3.iter() {  // Desreferenciamos (&i32 → i32)
```

```
        println!("{}", x);
    }

    println!("Array original: {:?}", arr3);
}
```

> "**`iter()`**" devuelve referencias `&T`, por lo que puedes leer los datos sin consumirlos.

Iteración con "`.iter_mut()`" por referencia mutable

Permite modificar los elementos del array durante la iteración.

```
fn main() {
    let mut arr4 = [1, 2, 3];

    println!("Iterando con iter_mut():");
    for x in arr4.iter_mut() {
        *x *= 10; // Desreferenciamos para modificar el valor
    }

    println!("Array modificado: {:?}", arr4);
}
```

> - `iter_mut()` devuelve `&mut T`.
> - Debes declarar el array como mut para usarlo.
> - Es la única forma segura de modificar los valores dentro del bucle for.

Iterando Vectores en Rust

En Rust, los vectores **`Vec<T>`** se recorren con bucles **for** de distintas maneras, dependiendo de cómo se accede a los elementos:

- Por valor **`for x in vec`**, consume el vector.
- Por referencia **`.iter()`**, solo lectura.
- Por referencia mutable **`.iter_mut()`**, permite modificar.
- Por propiedad **`.into_iter()`**, transfiere la propiedad de los elementos.

Iteración por valor `for x in vec`

Cuando usamos `for x in vec`, Rust mueve el vector.

Después del bucle, ya no puedes usarlo.

```
fn main() {
    let vec1 = vec![10, 20, 30];
```

```
    println!("Iterando con for in <valor>:");
    for x in vec1 {
        println!("{}", x);
    }

    // Error si intentas usar vec1 después:
    // println!("{:?}", vec1); // Moved value
}
```

> En vectores, a diferencia de los arrays, los valores no se copian automáticamente, se mueven.

Iteración con `.into_iter()`

`into_iter()` convierte el vector en un iterador que toma la propiedad de cada elemento.

```
fn main() {
    let vec2 = vec!["Rust", "es", "genial"];

    println!("→ Iterando con into_iter():");
    for palabra in vec2.into_iter() {
        println!("{}", palabra);
    }

    // El vector ya no está disponible:
    // println!("{:?}", vec2); // Error
}
```

> - Similar a `for x in vec`, pero más explícito.
> - Útil cuando deseas consumir el vector completamente, por ejemplo al mover datos.

Iteración con `.iter()` por referencia

`iter()` presta referencias a los elementos del vector, sin moverlo.

```
fn main() {
    let vec3 = vec![100, 200, 300];

    println!("Iterando con iter():");
    for valor in vec3.iter() {
        println!("{}", valor);
    }

    println!("Vector original sigue disponible: {:?}", vec3);
```

```
}
```

- El tipo de valor es &i32 “referencia”.
- Si quieres el valor directo, puedes desreferenciar: for &valor in **`vec3.iter() { ... }`**.

Iteración con `.iter_mut()` por referencia mutable

`iter_mut()` permite modificar los elementos directamente en el bucle.

```
fn main() {
    let mut vec4 = vec![1, 2, 3];

    println!("Iterando con iter_mut():");
    for valor in vec4.iter_mut() {
        *valor *= 10; // modificamos directamente
    }

    println!("Vector modificado: {:?}", vec4);
}
```

- El tipo de valor es `&mut i32`.
- Debes declarar el vector como `mut`.
- Los cambios se aplican directamente al vector original.

Iterar con índices cuando lo necesitas

A veces, también puedes iterar manualmente con índices si necesitas acceder a la posición.

```
fn main() {
    let vec5 = vec!["A", "B", "C"];

    println!("Iterando con índices:");
    for i in 0..vec5.len() {
        println!("Índice: {}, Valor: {}", i, vec5[i]);
    }
}
```

El acceso con **`vec[i]`** puede causar pánico si el índice está fuera del rango. Para evitarlo, usa **`vec.get(i)`** que devuelve **`Option<&T>`**.

For con Tuplas

Una tupla en sí no es iterable.

No puedes hacer directamente:

```
for x in (1, 2, 3) { } // Error
```

Esto se debe a que una tupla es una estructura de longitud fija y heterogénea, no una secuencia.

Sin embargo, puedes iterar colecciones que contienen tuplas, como vectores o arrays, y desestructurarlas dentro del for.

Ejemplo:

```
let pares = vec![(1, "uno"), (2, "dos"), (3, "tres")];

for (n, palabra) in pares {
    println!("{} -> {}", n, palabra); // consume pares
}
```

- **`for (n, palabra)`** in pares desestructura cada tupla del vector.
- El vector pares es consumido "moved" porque no usamos referencias.

Si queremos conservarlo:

```
let mapa = vec![(1, 2), (3, 4)];

for (a, b) in &mapa { // iterando por referencia
    println!("{} + {} = {}", a, b, a + b);
}
```

Aquí, a y b son referencias **`&i32`**, y el vector mapa sigue disponible después del bucle.

For con Strings (&str y String)

Las cadenas de texto pueden recorrerse de distintas formas, según qué desees iterar: caracteres, bytes o graphemes (combinaciones visuales de caracteres).

Ejemplo:

```
let s = "hola 😊";

for c in s.chars() {
    println!("{}", c);
}

for b in s.bytes() {
    println!("{}", b);
}
```

Detalles importantes:

- `for c in s.chars()`, itera sobre caracteres Unicode (char).

  Ejemplo de salida: `h`, `o`, `l`, `a`, , `😊`.

- `for b in s.bytes()`, itera sobre bytes **u8**, mostrando la codificación UTF-8.

  Ejemplo: `104`, `111`, `108`, `97`, `32`, `240`, `159`, `152`, `138`.

break y continue

- `break` : termina el bucle actual.
- `continue` : salta a la siguiente iteración.

```
for i in 0..6 {
    if i == 3 {
        continue; // se salta el 3
    }
    if i == 5 {
        break; // rompe el bucle
    }
    println!("{}", i);
}
```

Etiquetas en For

Rust permite etiquetar bucles para romper o continuar desde un nivel específico.

```
'outer: for i in 1..=3 {
    for j in 1..=3 {
        if i == 2 && j == 2 {
            break 'outer; // rompe el bucle externo
        }
        println!("i={}, j={}", i, j);
    }
}
```

Salida:

```
i=1, j=1
i=1, j=2
i=1, j=3
i=2, j=1
```

Ownership en for

Por defecto, for consume “mueve” la colección.

Para evitarlo, se itera por referencia `&` o `&mut`.

Ejemplo mutable:

```
fn main() {
    let mut vec = vec![1, 2, 3];
    for num in vec.iter_mut() {  // Itera con mut borrow
        *num += 1;
```

```
    }
    println!("{:?}", vec); // [2, 3, 4]
}
```

Iteradores Avanzados

Los iteradores son una parte esencial de Rust: permiten recorrer datos de forma inmutable, segura y funcional, sin necesidad de bucles imperativos.

Cualquier tipo que implemente el trait **Iterator** puede usarse en un **for**, o manipularse mediante sus métodos.

Creación de iteradores

```
let v = vec![10, 20, 30];
let mut it = v.iter(); // &i32

println!("{:?}", it.next()); // Some(&10)
println!("{:?}", it.next()); // Some(&20)
println!("{:?}", it.next()); // Some(&30)
println!("{:?}", it.next()); // None
```

Cada llamada a `.next()` devuelve `Option<T>`, hasta que se agotan los elementos.

Métodos comunes:

- **`map()`** : transforma

```
let v = vec![1, 2, 3];
let doblados: Vec<_> = v.iter().map(|x| x * 2).collect();
println!("{:?}", doblados); // [2, 4, 6]
```

- **`filter()`** : filtra según condición

```
let v = vec![1, 2, 3, 4, 5];
let pares: Vec<_> = v.iter().filter(|x| *x % 2 == 0).collect();
println!("{:?}", pares); // [2, 4]
```

- **`fold()`** : acumula (reduce)

```
let suma = (1..=5).fold(0, |acc, x| acc + x);
println!("{}", suma); // 15
```

- **`take()` `skip()`**

```
let v = vec![10, 20, 30, 40, 50];

for n in v.iter().skip(1).take(3) {
    println!("{}", n);
}
// Imprime: 20, 30, 40
```

- **`rev()`** : invierte el orden

```
for n in (1..=3).rev() {
    println!("{}", n);
}
// 3, 2, 1
```

Combinaciones poderosas

```
let datos = vec![1, 2, 3, 4, 5, 6];

let resultado: Vec<_> = datos
    .iter()
    .filter(|x| *x % 2 == 0) // pares
    .map(|x| x * x)          // al cuadrado
    .rev()                   // orden inverso
    .collect();

println!("{:?}", resultado); // [36, 16, 4]
```

Iterar múltiples colecciones

```
fn main() {
    let a = vec![1, 2, 3];

    let b = vec!["uno", "dos", "tres"];

    for (num, texto) in a.iter().zip(b.iter()) {  // Tupla (i32, &str)
        println!("{}: {}", num, texto);
    }

    // Salida: 1: uno, 2: dos, 3: tres
    // Se detiene en el más corto.
}
```

Búsqueda de Iteradores

Estos métodos trabajan sobre iteradores y devuelven valores booleanos o resultados parciales en lugar de colecciones completas.

No consumen todos los elementos si no es necesario, paran cuando encuentran el resultado.

- **`any()`**

  Devuelve true si al menos un elemento cumple la condición.

  Es equivalente a decir:

  “¿Existe alguno que cumpla esto?”

```
fn main() {
```

```
    let numeros = vec![1, 2, 3, 4, 5];

    let hay_par = numeros.iter().any(|&x| x % 2 == 0);
    println!("¿Hay algún número par? {}", hay_par); // true

    let mayor_a_diez = numeros.iter().any(|&x| x > 10);
    println!("¿Alguno mayor a 10? {}", mayor_a_diez); // false
}
```

> `any()` deja de iterar en cuanto encuentra un true.
>
> Ejemplo: no sigue buscando más pares cuando ya encontró uno.

- **`all()`**

  Devuelve true si todos los elementos cumplen la condición.

  Es equivalente a decir:

  "¿Todos cumplen esto?"

```
fn main() {
    let edades = vec![18, 22, 25, 30];

    let todos_mayores = edades.iter().all(|&x| x >= 18);
    println!("¿Todos son mayores de edad? {}", todos_mayores); // true

    let todos_pares = edades.iter().all(|&x| x % 2 == 0);
    println!("¿Todos son pares? {}", todos_pares); // false
}
```

> `all()` también detiene la iteración en cuanto encuentra un false.

- **`find()`**

  Busca el primer elemento que cumpla una condición y devuelve Option.

  Es equivalente a decir:

  "Encuentra el primer elemento que cumpla esto".

```
fn main() {
    let numeros = vec![10, 20, 30, 40, 50];

    if let Some(x) = numeros.iter().find(|&&n| n > 25) {
        println!("El primero mayor a 25 es: {}", x); // 30
    } else {
        println!("No hay ninguno mayor a 25");
```

```
    }

    // Otro ejemplo: buscar una palabra
    let palabras = vec!["rust", "golang", "python"];

    match palabras.iter().find(|&&p| p.starts_with('p')) {
        Some(p) => println!("Encontrado: {}", p),  // python
        None => println!("No encontrado"),
    }
}
```

- `find()` devuelve una referencia `Option<&T>` si usas `.iter()`.
- Si usas `.into_iter()`, moverá los elementos `Option<T>`.

Ejemplo combinado

Estos métodos se pueden combinar para expresiones potentes:

```
fn main() {
    let nums = vec![1, 2, 3, 4, 5];

    // ¿Existe algún impar mayor a 3?
    let resultado = nums.iter().any(|&x| x % 2 == 1 && x > 3);
    println!("¿Impar > 3?: {}", resultado); // true (5)

    // Verificar todos menores que 10
    println!("¿Todos < 10?: {}", nums.iter().all(|&x| x < 10)); // true

    // Buscar primer número divisible por 4
    if let Some(x) = nums.iter().find(|&&x| x % 4 == 0) {
        println!("Divisible por 4: {}", x); // 4
    }
}
```

Concepto clave

Los iteradores en Rust son lazy (perezosos): no ejecutan nada hasta que se consumen, por ejemplo con:

- collect()
- for
- count()
- sum()
- any(), all(), find()

Ejemplo:

```
let v = vec![1, 2, 3];
```

```
let it = v.iter().map(|x| x * 10); // aún no se ejecuta
println!("{}", it.sum::<i32>());   // se ejecuta aquí
```

Funciones en Rust

Las funciones son bloques de código reutilizables diseñados para realizar una tarea específica. Facilitan la modularidad, el mantenimiento y la reutilización del código al dividir un programa en partes más pequeñas y manejables. En Rust, las funciones se distinguen de los métodos que están asociados a structs o enums mediante impl, aquí nos enfocamos en funciones independientes.

Se declaran con la palabra clave `fn`, y Rust enfatiza la seguridad de tipos: los parámetros siempre requieren tipos explícitos, y los retornos son opcionales pero tipados.

Sintaxis básica:

```
fn nombre_funcion(param1: Tipo1, param2: Tipo2) -> TipoRetorno {
    // Cuerpo de la función
    // El último expresión sin ; se devuelve automáticamente
}
```

Ejemplo simple:

```
fn saludar() {
    println!("¡Hola, mundo!");
}
fn main() {
    saludar();  // Llama la función
}
```

Parámetros

Los parámetros son entradas a la función y deben especificar su tipo. Rust no infiere tipos en declaraciones de parámetros para evitar ambigüedades.

```
/// Suma dos números enteros y los imprime.
fn sumar(a: i32, b: i32) {
    println!("La suma es: {}", a + b);
}

fn main() {
    sumar(5, 3);  // Salida: La suma es: 8
}
```

Valores de retorno

Las funciones pueden devolver valores. Si no se especifica, devuelven "`() (unit type)`".

- Retorno implícito:

La última expresión sin **";"** se devuelve automáticamente, estilo idiomático de Rust.

```
fn cuadrado(x: i32) -> i32 {
    x * x  // Sin ";"  devuelve el resultado
}

fn main() {
    let resultado = cuadrado(4);
    println!("Cuadrado: {}", resultado);  // 16
}
```

Si agregas **";"**:

```
fn cuadrado_con_semi(x: i32) -> i32 {
    x * x;  // Con ";"  devuelve ()
    0       // Esto se ignora
}
```

return Explícito

Para salidas tempranas "early returns", usa **"`return expr;`"**. Es opcional al final, pero útil en condicionales.

```
fn main() {
    let a = 15;
    let b = 9;

    println!("El valor mayor de {a} y {b} es : {}", valor_mayor(a , b));
}

fn valor_mayor(a:i32, b:i32) -> i32{
    if a > b {
        return a
    }else{
        return b
    }
}
```

Otro ejemplo: Verificar paridad.

```
fn es_par(n: i32) -> bool {
    if n % 2 == 0 {
        true  // Expresión
    } else {
        false
    }
}
```

```rust
/// Equivalente con return (más conciso).
fn es_par_ret(n: i32) -> bool {
    return n % 2 == 0;  // Explícito
}
```

Funciones anidadas

Rust permite definir funciones dentro de otras "**inner functions**", pero solo visibles en el scope padre. Útiles para helpers locales.

```rust
fn principal() {
    fn interna(x: i32) -> i32 {  // Anidada
        x + 1
    }
    println!("{}", interna(5));  // 6
}

fn main() {
    principal();
}
```

Closures: Funciones Anónimas

Los closures son funciones anónimas que capturan el entorno "**variables externas**". Son como lambdas en Python/JS, pero con reglas estrictas de captura "**borrow/move**". Ideales para iteradores, callbacks y FP.

Sintaxis:

```rust
let nombre_closure = |param1: Tipo1, param2: Tipo2| -> TipoRetorno {
    // Cuerpo (expresión o bloque)
};
```

Ejemplo básico:

```rust
let resta = |a: i32, b: i32| -> i32 {a - b};  // Explícito
let suma  = |a: i32, b: i32| a + b;  // Infieren tipos si posible
println!("{}", suma(2, 3));         // 5
println!("{}", resta(5, 3));         // 2
```

Devolviendo Múltiples Valores: Tuplas

Las funciones pueden devolver múltiples valores empaquetados en una tupla:

```rust
fn calcular(a: i32, b: i32) -> (i32, i32) {
    (a + b, a * b)  // Tupla como último valor
}
```

```rust
fn main() {
    let (suma, producto) = calcular(4, 5);
    println!("Suma: {}, Producto: {}", suma, producto);  // 9, 20
}
```

Devolver un array

Los arrays en Rust tienen tamaño fijo que se define en el tipo.

```rust
fn crear_array() -> [i32; 3] {
    [10, 20, 30]
}

fn main() {
    let arr = crear_array();
    println!("Array: {:?}", arr);  // [10, 20, 30]
}
```

Devolver un vector

Un vector `Vec<T>` es dinámico y mucho más usado en la práctica:

```rust
/// Crea un vector de strings.
fn crear_vector() -> Vec<String> {
    vec!["Rust".to_string(), "es".to_string(), "genial".to_string()]
}

fn main() {
    let v = crear_vector();
    println!("Vector: {:?}", v);  // ["Rust", "es", "genial"]
}
```

Funciones con referencias

Pasa datos grandes por &T para evitar copias/moves.

```rust
/// Calcula longitud sin tomar ownership.
fn longitud(texto: &String) -> usize {  // &String (o &str para más flex)
    texto.len()
}

fn main() {
    let s = String::from("Hola Rust");
    let l = longitud(&s);  // Borrow
    println!("'{}' tiene {} caracteres", s, l);  // s intacto
}
```

Funciones por referencia y mutables

```rust
fn incrementar(x: &mut i32) {
    *x += 1; // se desreferencia con *
}

fn main() {
    let mut num = 10;
    incrementar(&mut num);
    println!("{}", num); // 11
}
```

Explicación rápida:

- **"`&mut`"**, referencia mutable (permite modificar la variable original).
- **"`*x`"**, desreferencia: accede al valor real dentro de la referencia.

Ejemplos prácticos de funciones con **`&mut`**

Ahora veamos casos más útiles y aplicables.

Ejemplo 1: Modificar un vector desde una función

Supongamos que queremos agregar elementos a un vector existente.

```rust
fn agregar_elemento(v: &mut Vec<i32>, nuevo: i32) {
    v.push(nuevo);
}

fn main() {
    let mut numeros = vec![1, 2, 3];
    agregar_elemento(&mut numeros, 4);
    agregar_elemento(&mut numeros, 5);

    println!("{:?}", numeros); // [1, 2, 3, 4, 5]
}
```

Aquí:

- Modifica directamente el vector original gracias a la referencia mutable.
- Sin &mut, el push() fallaría porque Rust no permitiría modificarlo.

Ejemplo 2: Convertir texto a mayúsculas

```rust
fn a_mayusculas(texto: &mut String) {
    *texto = texto.to_uppercase();
}

fn main() {
    let mut saludo = String::from("hola rust");
    a_mayusculas(&mut saludo);
    println!("{}", saludo); // HOLA RUST
```

```
}
```

Explicación:

- `to_uppercase()` crea un nuevo `String`.
- Lo reasignamos sobre el mismo valor usando "**`*texto = ...`**"

Ejemplo 3: Restablecer valores en un arreglo mutable

```
fn resetear(arr: &mut [i32]) {
    for x in arr.iter_mut() {
        *x = 0;
    }
}

fn main() {
    let mut datos = [3, 7, 9];
    resetear(&mut datos);
    println!("{:?}", datos); // [0, 0, 0]
}
```

Puntos clave:

- `&mut [i32]` es un slice mutable del array.

Ejemplo 4: Intercambiar valores (swap)

```
fn intercambiar(a: &mut i32, b: &mut i32) {
    let temp = *a;
    *a = *b;
    *b = temp;
}

fn main() {
    let mut x = 10;
    let mut y = 20;
    intercambiar(&mut x, &mut y);
    println!("x = {}, y = {}", x, y); // x = 20, y = 10
}
```

Útil cuando quieres manipular variables sin devolver múltiples valores.

Funciones genéricas

Una función genérica permite que el tipo de sus parámetros no esté fijo.

En lugar de hacer una función solo para `i32` o `f64`, puedes hacer una que funcione con cualquier tipo.

Ejemplo sin genéricos

```
fn mostrar_i32(x: i32) {
```

```
    println!("El valor es: {}", x);
}

fn main() {
    mostrar_i32(10);
}
```

Esta función solo acepta `i32`.

Si intentas pasarle un `f64`, no compila.

Ejemplo genérico básico

```
fn mostrar<T>(x: T) {
    println!("Tengo un valor!");
}

fn main() {
    mostrar(10);        // i32
    mostrar(3.14);      // f64
    mostrar("hola");    // &str
}
```

Retorno genérico

```
fn devolver_mismo<T>(x: T) -> T {
    x
}

fn main() {
    let n = devolver_mismo(5);

    let t = devolver_mismo("texto");

    println!("{} - {}", n, t);
}
```

Esta función devuelve exactamente el mismo tipo que recibe.

Ejemplo 1: Intercambiar valores

Una función genérica puede manipular cualquier tipo, no solo devolverlo.

Veamos cómo intercambiar dos valores de cualquier tipo (sin importar si son i32, String, etc.):

```
fn intercambiar<T>(a: &mut T, b: &mut T) {
    let temp = *a;      // mueve el valor de a
    *a = *b;            // reemplaza a por b
    *b = temp;          // reemplaza b por temp
}
```

```rust
fn main() {
    let mut x = 10;
    let mut y = 20;

    intercambiar(&mut x, &mut y);
    println!("x = {}, y = {}", x, y); // x = 20, y = 10
}
```

Aquí:

- `T` puede ser cualquier tipo (numérico, texto, struct...).
- `&mut T` indica que tomamos referencias mutables (podemos modificarlas).
- No necesitamos saber qué tipo es `T`, Rust se encarga en tiempo de compilación.

Ejemplo 4: Genéricos + funciones que retornan otro tipo genérico

```rust
fn convertir_en_vec<T>(x: T) -> Vec<T> {
    vec![x]
}

fn main() {
    let numeros = convertir_en_vec(42);
    let palabras = convertir_en_vec("hola");

    println!("{:?}", numeros);  // [42]
    println!("{:?}", palabras); // ["hola"]
}
```

Esta función:

- Toma un valor de tipo T
- Devuelve un `Vec<T>` (vector del mismo tipo)

Struct

En Rust, un struct '**estructura**' es una forma de agrupar múltiples valores relacionados bajo un tipo de dato unificado. Funciona como un contenedor organizado que nos permite definir nuestro propio tipo de dato personalizado.

Por ejemplo, imagina un Libro como una ficha bibliográfica: en lugar de manejar datos sueltos sobre un libro, reunimos toda la información relevante como el título, autor y año de publicación en un solo registro con campos específicamente etiquetados.

Esto no solo mantiene los datos lógicamente unidos, sino que también le da al compilador de Rust la capacidad de verificar que accedemos a campos válidos, como titulo o autor. Si intentas acceder a un campo no definido, como editorial, Rust te avisará de ese error en tiempo de compilación.
Además, al asignar tipos correctos a cada campo como por ejemplo, String para el título y u32 para el año, prevenimos errores comunes antes de que el programa se ejecute, en lugar de descubrirlos durante la ejecución. Esto hace que el código sea más robusto y fácil de mantener.

:: Sintaxis básica

Un struct se declara con la palabra clave `struct`, seguida del nombre y una lista de campos entre llaves:

```
struct Libro {
    titulo: String,
    autor: String,
    anio_publicacion: u32,
}
```

- `Libro` es el nombre del `struct`.
- titulo, autor y anio_publicacion son los campos.
- Cada campo tiene un nombre y un tipo.
- Los nombres deben usar `PascalCase`, según la convención oficial de Rust.

:: Instanciación de un Struct

Para crear una instancia del `struct`, se usa las llaves `{ }` indicando los valores de cada campo.

```
fn main() {
    let libro = Libro {
        titulo: String::from("Domina Rust"),
        autor: String::from("Alex Villanueva"),
        anio_publicacion: 2025,
    };
}
```

Reglas importantes:

- Debes inicializar todos los campos.
- El orden no importa.
- Puedes crear structs como let o let mut.

:: Acceso a los campos

Los campos se acceden con el operador `.`:

```
  // --snip--

  println!("Título: {}", libro.titulo);   // "Domina Rust"
  println!("Autor: {}", libro.autor);     // "Alex Villanueva"
  println!("Año: {}", libro.anio_publicacion);  // 2025
```

:: Structs Mutables

Para modificar los campos de un struct, la instancia debe ser mutable `mut`:

```
let mut libro = Libro {
    // --snip--
};

libro.anio_publicacion = 2026; // Modificamos un campo
println!("Año: {}", libro.anio_publicacion);  // Ahora 2026
```

Si la variable no es mutable, sus campos tampoco lo serán.

:: Struct Update Syntax

Puedes crear una nueva instancia copiando algunos campos de otro struct con la sintaxis “`..`”:

```
// --snip--

let libro2 = Libro {
    titulo: String::from("Aprende RusT"),
    ..libro
};

println!("{}", libro2.titulo); // "Aprende RusT"
println!("{}", libro2.autor); // "Alex Villanueva"
println!("{}", libro2.anio_publicacion);  // 2026
```

Esto copia:

- libro.autor
- libro.anio_publicacion

Si un campo contiene datos que mueven ownership como String, la variable original deja de ser válida para ese campo.

:: Tuple Structs

Son structs sin nombre para los campos. Útiles cuando el significado viene implícito:

```
struct Punto(i32, i32);
```

```
let p = Punto(10, 20);
```

:: Unit-like Structs

No tienen campos. Se usan para:

- Implementar traits en tipos sin datos
- Marcar eventos o comportamientos
- Crear tipos fantasma (zero-sized types)

```
struct SinDatos;

let x = SinDatos;
```

:: Desestructuración de Structs

Podemos extraer los campos de un struct en variables individuales:

```
// --snip--

let Libro { titulo, autor, anio_publicacion } = libro2;
println!("{} - {} ({})", titulo, autor, anio_publicacion);
```

:: Derivaciones automáticas

Rust puede generar implementaciones comunes automáticamente con `#[derive(...)]`.

```
#[derive(Debug, Clone, PartialEq)]
struct Libro {
    // --snip--
}

fn main() {
    // --snip--
    let Libro { titulo, autor, anio_publicacion } = libro2.clone();
    // --snip--
    println!("{:?}", libro); // Usa Debug
    println!("{:#?}", libro); // Usa Debug

    let libro3 = Libro{
        titulo: String::from("EFFECTIVE RUST"),
        autor: String::from("David Drysdale"),
        anio_publicacion: 2000,
    };
```

```
    println!("{}",libro2 == libro3); // False
}
```

- `Debug`: permite imprimir structs con {:?}.
- `Clone`: permite clonar valores.
- `PartialEq`: permite comparar con ==.

:: Métodos en Structs

En Rust, cuando queremos añadir comportamiento a nuestros structs, utilizamos el bloque `impl` para implementar métodos.

Esto transforma nuestros structs de ser simples contenedores de datos estaticos a tipos de datos inteligentes que saben cómo operar con su propia información, manteniendo todo el código relacionado organizado en un mismo lugar y aprovechando el sistema de tipos de Rust para garantizar que estas operaciones se realicen de manera segura.

Sintaxis:

```
impl Libro {
    // Aquí van los métodos y funciones asociadas
}
```

Ejemplo básico:

```
// --snip--
impl Libro {
  fn new(titulo: String, autor: String, anio_publicacion: u32) -> Libro {
    Libro {
      titulo,
      autor,
      anio_publicacion
    }
  }

}

fn main() {
  // --snip--
  let libro4 = Libro::new(
    String::from("Domina Rust"),
    String::from("Alex Villanueva"),
    2025
  );
  // Usa :: para métodos asociados
```

```
}
```

:: Self

Self representa el tipo del `struct` dentro del bloque `impl`.

Es una forma abreviada de escribir el nombre completo del tipo. `Self = Libro`.

```
fn new(..) -> Self {
  Self {
  ...
  }
}
```

```
fn new(..) -> Libro {
  Libro {
  ...
  }
}
```

Las Variantes de self

Todos los métodos de instancia usan `self` como primer parámetro. Las variantes controlan ownership y mutabilidad. En Rust, los métodos dentro de impl pueden recibir al propio objeto de diferentes formas.

1. Borrow Inmutable `self`
   - Referencia inmutable al objeto actual.
   - No puede modificar los campos, solo leerlos.
   - Se traduce a: `self: &Self`.

   Ejemplo:

```
impl Libro {
    fn es_reciente(&self) -> bool {  // &self: solo lee
        self.anio_publicacion >= 2020
    }
}
```

   Forma abreviada de escribir:

```
fn es_reciente(&self)  ->  fn es_reciente(self: &Libro)
```

```
fn main() {
    // --snip--

  let libro4 = Libro::nuevo(
    String::from("Nuevo Libro"),
    String::from("Autor X"),
    2025
  );

  println!("¿Es reciente? {}", libro4.es_reciente());  // true
  // libro sigue intacto, puedes usarlo después
```

```
  println!("Año: {}", libro4.anio_publicacion);  // 2026
}
```

2. Borrow Mutable `&mut self`.

   - Modifica y lee los campos.
   - Solo una mutación a la vez.

   Ejemplo:

```
impl Libro {
    fn actualizar_año(&mut self, nuevo_año: u32) {  // &mut self: modifica
        self.anio_publicacion = nuevo_año;
    }
}
```

   Forma abreviada de escribir:

```
fn actualizar_año(&mut self,..)  ->  fn actualizar_año(self: &mut Libro,..)
```

   Necesitas

   - `let mut libro`
   - `&mut self`.

```
fn main() {
    // --snip--

  libro4.actualizar_año(2027);  // Cambia vía método
  println!("Año actualizado: {}", libro.anio_publicacion);  // 2027
  // libro mut sigue vivo y modificado
}
```

3. Tomar Ownership `self`

   Mueve la instancia al método **ownership transferido**.

   Después, no puedes usar el struct original.

   Ejemplo:

```
impl Libro {

    fn a_string(self) -> String {  // self: consume
        format!("{} por {} ({})",
          self.titulo,
          self.autor,
          self.anio_publicacion
        )
    }
}
```

```
fn main() {
    // --snip--

    let descripcion = libro4.a_string();  // Consume libro4
    println!("{}", descripcion);  // Domina Rust por Alex Villanueva (2027)
    //println!("{:?}", libro4);  // Error: moved!
}
```

4. `&Self` & `&mut Self`

   Se usan normalmente en las firmas de métodos estáticos o asociados, o cuando defines tipos o traits genéricos, donde Self hace que tu código sea más genérico y expresivo.

```
impl Libro{

    fn comparar(&self, otro: &Self) -> bool {   // &Self = &Libro
        self.titulo == otro.titulo
    }

    fn incrementar(&mut self, otro: &mut Self) {
        otro.anio_publicacion += self.anio_publicacion;
    }
}
```

   Uso:

```
fn main(){
  // --snip--

  libro_y.incrementar(&mut libro_x);
  println!("{}", libro_y.anio_publicacion);

  println!("{}",libro_y.comparar(&libro_x));
}
```

:: Method Chaining

En Rust, es posible llamar varios métodos de forma consecutiva sobre el mismo valor usando un estilo fluido conocido como encadenamiento de métodos “method chaining”.

Este patrón es muy común en APIs modernas, ya que hace el código más expresivo y fácil de leer.

```
persona
    .incrementar(1)
    .incrementar(2)
    .resetear();
```

Este estilo solo es posible cuando los métodos están implementados dentro de un bloque `impl`.

¿Qué permite el encadenamiento de métodos?

Para encadenar métodos, un método debe retornar `Self`, `&Self` o `&mut Self`.

Es decir, debe devolver la propia instancia para poder llamar al siguiente método sobre ella.

Rust no tiene un "azúcar sintáctico especial" para esto; es solo una consecuencia natural de devolver el mismo objeto.

Ejemplo básico:

```
struct Persona {
    edad: u8,
}

impl Persona {
    fn new(edad: u8) -> Self {
        Self { edad }
    }

    // Permite encadenar gracias a Self
    fn sumar(mut self, x: u8) -> Self {
        self.edad += x;
        self
    }

    // También retorna Self
    fn duplicar(mut self) -> Self {
        self.edad *= 2;
        self
    }

    fn mostrar(&self) {
        println!("Edad: {}", self.edad);
    }
}

fn main() {
    let persona = Persona::new(20)
        .sumar(5)       // 25
        .duplicar();    // 50

    persona.mostrar();
}
```

Salida:

```
Edad: 50
```

Enums y Patrones

Rust ofrece una poderosa combinación entre enumeraciones “enums” y el sistema de patrones “pattern matching”.

Juntos permiten representar datos complejos de forma segura y expresiva.

Enum

Un enum te permite definir un tipo que puede ser uno entre varios valores posibles.

Ejemplo basico:

```
enum Color {
    Rojo,
    Verde,
    Azul,
}
```

Los enum son muy útiles para representar estados, opciones y variaciones de datos.

Enums con Datos

A diferencia de otros lenguajes, Rust permite que cada variante del enum tenga datos asociados.

- Con valores simples

```
enum Ip {
    V4(String),
    V6(String),
}
```

- Con estructuras diferentes

```
enum Mensaje {
    Texto(String),
    Coordenada { x: i32, y: i32 },
    Salir,
}
```

match: La herramienta principal para enums

match permite comparar un valor contra patrones.

Ejemplo básico:

```
fn imprimir_color(color: Color) {
    match color {
        Color::Rojo => println!("Rojo!"),
        Color::Verde => println!("Verde!"),
        Color::Azul => println!("Azul!"),
    }
}
```

Patrones con Enums que tienen datos

- Extraer valores

```
enum Mensaje {
    Texto(String),
    Numero(i32),
}

fn procesar(m: Mensaje) {
    match m {
        Mensaje::Texto(t) => println!("Texto: {}", t),
        Mensaje::Numero(n) => println!("Número: {}", n),
    }
}
```

- Con estructura

```
enum Comando {
    Mover { x: i32, y: i32 },
}

fn ejecutar(c: Comando) {
    match c {
        Comando::Mover { x, y } => {
            println!("Moviendo a la posición ({}, {})", x, y);
        }
    }
}
```

Patrones con rangos

```
enum Evento {
    Nivel(i32),
}

match Evento::Nivel(5) {
    Evento::Nivel(n @ 1..=10) => println!("Nivel bajo: {}", n),
    Evento::Nivel(n) => println!("Nivel alto: {}", n),
}
```

Enums con Métodos

Los enums también pueden tener métodos.

```
enum Estado {
    Online,
    Offline,
}
```

```
impl Estado {
    fn esta_activo(&self) -> bool {
        matches!(self, Estado::Online)
    }
}
```

Uso:

```
let e = Estado::Online;

println!("{}", e.esta_activo()); // true
```

Imprimir Enum

```
#[derive(Debug)]
enum Direccion {
    Norte,
    Sur,
    Este,
    Oeste,
}

fn main() {
    let dir = Direccion::Norte;
    println!("{:?}", dir);
}
```

Generic Types

Los genéricos permiten escribir código que funciona con múltiples tipos sin duplicar lógica. Son fundamentales en Rust porque habilitan:

- Reutilización de código
- Flexibilidad sin perder seguridad
- Rendimiento máximo gracias a la monomorfización

¿Qué son los genéricos?

Un generic type es un "comodín de tipos" que le permite al compilador crear múltiples versiones de una misma estructura, enum o función, una por cada tipo concreto que uses, sin que tengas que escribir el código una y otra vez.

- En lugar de decir:

  "Esta estructura solo sirve para i32"

- Dices:

> "Esta estructura sirve para cualquier tipo y le pondré de nombre T y tú como compilador, generas la versión correcta cuando se use con i32, String, User, etc."

Sintaxis:

```
fn identidad<T>(valor: T) -> T {
    valor
}
```

<T> es un parámetro de tipo genérico. Esto significa que la función funciona para:

- i32, String, bool etc
- incluso para tipos definidos por el usuario.

Ejemplo:

```
let n = identidad(10);
let s = identidad("hola");
let b = identidad(true);
```

Rust genera una versión optimizada para cada tipo utilizado **monomorfización**.

Genéricos en estructuras

Puedes declarar un struct que acepte tipos genéricos:

```
struct Punto<T> {
    x: T,
    y: T,
}
```

Uso:

```
let p1 = Punto { x: 10, y: 20 };
let p2 = Punto { x: 3.5, y: 6.1 };
let p3 = Punto { x: "hola", y: "mundo" };
```

Incluso diferentes tipos en un mismo struct:

```
struct Par<T, U> {
    a: T,
    b: U,
}

let par = Par { a: "edad", b: 24 };
```

Métodos genéricos `impl<T>`

También puedes definir métodos dentro de un impl genérico:

```
struct Contenedor<T> {
    valor: T,
}
```

```
impl<T> Contenedor<T> {
    fn obtener(&self) -> &T {
        &self.valor
    }
}
```

Incluso métodos adicionales para un tipo concreto:

```
impl Contenedor<i32> {
    fn es_par(&self) -> bool {
        self.valor % 2 == 0
    }
}
```

Funciones con múltiples genéricos

Una función puede tener dos, tres o más tipos genéricos, cada uno con sus propias restricciones.

```
fn mezclar<T, U, V>(a: T, b: U, c: V) -> (T, U, V) {
    (a, b, c)
}
```

Traits

Un trait es un conjunto de métodos que un tipo puede implementar. Es similar a interfaces en otros lenguajes, pero más poderoso.

Sirve para:

- Definir comportamientos comunes para distintos tipos.
- Permitir polimorfismo.
- Exigir que un tipo tenga cierto método para poder usarse en una función genérica.
- Poder escribir código genérico sin importar el tipo mientras cumpla el trait.

Traits Comunes de la Librería Estándar

Antes de crear tus propios traits, es importante conocer algunos de los traits más utilizados que vienen incluidos en la Standard Library.

Muchos tipos los implementan automáticamente y aparecen con frecuencia en código real.

1. Trait Debug

   Permite imprimir un valor para depuración con `{:?}` o `{:#?}`.

```
#[derive(Debug)]
struct Persona {
    nombre: String,
    edad: u32,
```

```
}

fn main() {
    let p = Persona { nombre: "Ana".into(), edad: 30 };
    println!("{:?}", p);
}
```

Útil para inspeccionar valores rápidamente.

1. Trait Clone

   Permite clonar un valor: crear una copia profunda

   ```
   #[derive(Clone)]
   struct Punto {
       x: i32,
       y: i32,
   }

   let a = Punto { x: 1, y: 2 };
   let b = a.clone();
   ```

   clone() crea una copia real del contenido.

2. Trait Copy

   Una extensión de Clone para tipos livianos y triviales de copiar.

   Ejemplos:

   - enteros
   - booleanos
   - caracteres
   - flotantes
   - tuplas pequeñas con solo tipos Copy

   ```
   #[derive(Copy, Clone)]
   struct Coordenada(i32, i32);

   let a = Coordenada(1,2);
   let b = a; // copia automática
   ```

3. Trait PartialEq

   Permite comparar valores con == y !=.

   ```
   #[derive(PartialEq)]
   struct Usuario {
       id: u32,
   }
   ```

```
let a = Usuario { id: 1 };
let b = Usuario { id: 1 };

assert!(a == b);
```

4. Trait Eq

   Indica que la igualdad es "total":

   Casi siempre aparece junto a PartialEq.

```
#[derive(PartialEq, Eq)]
struct Dni(u32);
```

5. Trait PartialOrd y Ord

   Permite usar <, >, <=, >= y ordenar colecciones.

```
#[derive(PartialOrd, Ord, PartialEq, Eq)]
struct Numero(i32);

let a = Numero(3);
let b = Numero(5);

assert!(a < b);
```

   Ord permite ordenamiento total (necesario para sort()).

Definición básica de un Trait

```
trait Saludo {
    fn saludar(&self) -> String;
}
```

Aquí no hay implementación, solo se define el requisito:

Si un tipo implementa `Saludo`, debe tener el método saludar.

Implementando un Trait en un Struct

```
struct Persona {
    nombre: String,
}

impl Saludo for Persona {
    fn saludar(&self) -> String {
        format!("Hola, soy {}", self.nombre)
    }
}
```

uso:

```
let p = Persona { nombre: "Luis".into() };
```

```
println!("{}", p.saludar());
```

Un Trait como Interface de Comportamiento

Puedes tener muchos structs diferentes con el mismo trait:

```
struct Robot {
    id: u32,
}

impl Saludo for Robot {
    fn saludar(&self) -> String {
        format!("Unidad {} reportándose", self.id)
    }
}
```

Así tienes polimorfismo:

```
fn presentar(s: &impl Saludo) {
    println!("{}", s.saludar());
}

presentar(&Persona { nombre: "Ana".into() });
presentar(&Robot { id: 42 });
```

Sintaxis alternativa de generics para Traits

Estas dos formas son equivalentes:

1. Forma corta (impl Trait)

   ```
   fn presentar(s: &impl Saludo) { ... }
   ```

2. Forma larga (generics + trait bound)

   ```
   fn presentar<T: Saludo>(s: &T) { ... }
   ```

Las dos hacen lo mismo:

permiten que la función acepte cualquier tipo que implemente el trait.

Traits con Métodos por Defecto

Un trait puede contener implementaciones por defecto:

```
trait Animal {
    fn sonido(&self) -> &str;

    fn hablar(&self) {
        println!("El animal hace: {}", self.sonido());
    }
}
```

Si tu struct implementa Animal pero no hablar, la versión por defecto se usará.

Traits con métodos estáticos

Los traits pueden tener métodos que no reciben self:

```
trait Identificable {
    fn tipo() -> &'static str;
}
```

Implementar un Trait genérico

Puedes usar generics dentro de los traits:

```
trait Sumar<T> {
    fn sumar(&self, otro: T) -> T;
}
```

Esto permite traits más flexibles.

Traits en Enums

Sí, los enums pueden implementar traits igual que los structs:

```
enum Estado {
    Activo,
    Inactivo,
}

impl Saludo for Estado {
    fn saludar(&self) -> String {
        match self {
            Estado::Activo => "Estoy activo".into(),
            Estado::Inactivo => "Estoy inactivo".into(),
        }
    }
}
```

Supertraits

Un trait puede depender de otro:

```
trait A: B { }
```

:: Trait Bounds

Una restricción que indica que un tipo genérico debe implementar un trait específico.

Sin bounds, un genérico no puede usar ninguna funcionalidad del trait.

Sintaxis básica:

```
fn mostrar<T: Debug>(x: T) {
    println!("{:?}", x);
}
```

Múltiples bounds

```
fn procesar<T: Clone + PartialEq>(x: T, y: T) -> bool {
    x == y
}
```

Sintaxis alternativa con where

Útil cuando hay varios bounds:

```
fn combinar<T, U>(t: T, u: U)
where
    T: Debug,
    U: Clone + Debug,
{
    println!("{:?}", t);
}
```

Bounds en structs

```
struct Punto<T: Copy> {
    x: T,
    y: T,
}
```

Bounds en impl

```
impl<T: PartialOrd> Maximo<T> {
    fn max(a: T, b: T) -> T {
        if a > b { a } else { b }
    }
}
```

Bounds en enums

```
enum Contenedor<T: Debug> {
    Valor(T),
}
```

Bounds súper-traits Un trait que requiere otro trait:

```
trait Imprimible: Display {
    fn imprimir(&self) {
        println!("{}", self);
    }
}
```

Lifetimes

Un lifetime es una etiqueta que describe cuánto tiempo vive una referencia.

No es un valor, no se almacena en memoria, no existe en tiempo de ejecución.

Solo es información estática para que el compilador asegure que no haya referencias colgantes.

Ejemplo de referencia peligrosa:

```
let r;

{
    let x = 10;
    r = &x;     // x muere aquí
}

println!("{}", r); // r apunta a algo destruido
```

El compilador detecta este tipo de errores antes de que el programa se ejecute.

Notación ‘a

Una lifetime se escribe con un apóstrofo:

```
&'a i32
```

Se lee:

- “referencia a un i32 que vive al menos tanto como ‘a”.

¿Por qué las funciones necesitan lifetimes explícitas?

Cuando una función recibe y devuelve referencias, Rust no sabe por sí solo qué referencia vive más.

Ejemplo:

```
fn mayor(a: &str, b: &str) -> &str {
    if a.len() > b.len() { a } else { b }
}
```

Esto produce error:

“la referencia devuelta necesita un lifetime, pero no está claro cuál”.

Para solucionarlo:

```
fn mayor<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() > b.len() { a } else { b }
}
```

Significado:

- ambos parámetros viven al menos ‘a
- el retorno también vive ‘a
- por lo tanto, el retorno es válido

Ejemplo completo de Lifetimes en funciones

```
fn el_mas_largo<'a>(x: &'a str, y: &'a str) -> &'a str {
```

```
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

uso:

```
let s1 = String::from("hola");
let s2 = String::from("rustaceo");

let res = el_mas_largo(s1.as_str(), s2.as_str());

println!("{}", res);
```

Lifetimes en Structs

Cuando un struct contiene referencias, debe tener lifetimes explícitas.

```
struct Texto<'a> {
    contenido: &'a str,
}

let s = String::from("Hola");
let t = Texto { contenido: &s };
```

El struct vive menos o igual que ‘a.

Lifetimes con métodos

```
struct Contenedor<'a> {
    valor: &'a str,
}

impl<'a> Contenedor<'a> {
    fn obtener(&self) -> &'a str {
        self.valor
    }
}
```

El ‘a del método coincide con el ‘a del struct.

Lifetimes en Enum

```
enum Mensaje<'a> {
    Texto(&'a str),
    Numero(i32),
}
```

Funcionan igual que en estructuras.

Reglas de Lifetime Elision

Rust aplica estas reglas cuando puede inferir sin ambigüedad:

Regla 1

Cada parámetro de entrada obtiene su propio lifetime.

```
fn f(x: &i32, y: &i32);
```

Se convierte en:

```
fn f<'a, 'b>(x: &'a i32, y: &'b i32);
```

Regla 2

Si hay un solo parámetro de entrada, el retorno recibe su lifetime.

```
fn id(x: &i32) -> &i32;  // lifetime elision
```

Equivalente a:

```
fn id<'a>(x: &'a i32) -> &'a i32;
```

Regla 3

Si el método tiene &self o &mut self, el retorno hereda ese lifetime.

```
fn valor(&self) -> &str;
```

Equivalente a:

```
fn valor<'a>(&'a self) -> &'a str;
```

Lifetime Bounds Puedes usar lifetimes como restricciones:

```
fn comparar<'a, T: PartialEq + 'a>(x: &'a T, y: &'a T) -> bool {
    x == y
}
```

T: ‘a significa:

- “T vive al menos tanto como ‘a”.

Múltiples lifetimes

Puede haber varias lifetimes independientes:

```
fn elegir<'a, 'b>(a: &'a str, b: &'b str) -> &'b str {
    b
}
```

Significa:

- ‘a y ‘b no dependen uno del otro
- el retorno depende de ‘b

Static lifetime ‘static

Es el lifetime más largo posible.

Significa:

- vive toda la duración del programa

```
let x: &'static str = "cadena literal";
```

Los literales string siempre son ‘static.

Ejemplo Real: dividir texto en partes

```
fn primera_palabra<'a>(s: &'a str) -> &'a str {
    let bytes = s.as_bytes();

    for (i, &c) in bytes.iter().enumerate() {
        if c == b' ' {
            return &s[..i];
        }
    }

    s
}
```

Este patrón aparece muchísimo:

función lee referencia y devuelve una subreferencia válida ⟶ lifetime ‘a.

Lifetimes y Traits juntos

Puedes tener traits parametrizados con lifetimes:

```
trait Logger<'a> {
    fn log(&self, msg: &'a str);
}
```

O implementar traits para tipos con lifetimes:

```
impl<'a> Default for Texto<'a> {
    fn default() -> Self {
        Self { contenido: "" }
    }
}
```

Bounds con lifetimes

```
fn mostrar<'a, T>(x: &'a T)
where
    T: Debug + 'a,
{
    println!("{:?}", x);
}
```

Manejo de Errores

:: Errores Recuperables

Rust trata el manejo de errores como una parte esencial del lenguaje.

Para evitar fallos inesperados, Rust propone una estrategia distinta a otros lenguajes: tratar los errores como valores, no como excepciones.

¿Qué es un error recuperable?

Un error recuperable es aquel que:

- Podemos anticipar
- Podemos manejar
- No requiere terminar el programa

Ejemplos:

- Un archivo que no existe
- Una conexión de red que falla
- Un número que no se puede convertir
- Un usuario que podría o no estar registrado

Rust usa `Option` y`Result` para estos casos.

▶ Option<T>

valor o ausencia de valor

```
enum Option<T> {
    Some(T),
    None,
}
```

Sirve para modelar cosas que pueden o no existir.

Obtener el primer elemento de un vector

```
fn obtener_primero(nums: &Vec<i32>) -> Option<i32> {
    nums.first().copied()
}
```

Uso:

```
match obtener_primero(&vec![10, 20, 30]) {
    Some(n) => println!("Primer número: {}", n),
    None => println!("El vector está vacío"),
}
```

Métodos importantes de Option<T>

- .unwrap()

  Obtiene el valor o hace panic

- .unwrap_or(x)

  Valor por defecto

- .unwrap_or_else()

  Llama una función para obtener un valor

- .map()

  Transforma Some

- .and_then()

  Encadena operaciones que también devuelven Option

Ejemplo: evitar panic usando unwrap_or

```
let nombre = Some("Juan");
let n = nombre.unwrap_or("Invitado");
```

▸ Result<T, E>

operación que puede fallar

Definición:

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Ejemplos: abrir archivos, parsear strings, convertir tipos.

Convertir un string en número

```
fn convertir(s: &str) -> Result<i32, std::num::ParseIntError> {
    s.parse::<i32>()
}
```

Uso:

```
match convertir("42") {
    Ok(n) => println!("Número: {}", n),
    Err(e) => println!("Error: {}", e),
}
```

Métodos importantes de Result<T, E>

- .unwrap()

  Obtiene T o panic

- .expect(msg)

  Igual que unwrap pero con mensaje

- .map()

  Transforma Ok

- .map_err()

  Transforma Err

- .ok()

  Convierte Result a Option

- .err()

  Obtiene la parte Err

▸ El operador ?

El operador ?:

- Si es Ok, devuelve el valor
- Si es Err, retorna inmediatamente de la función

Ejemplo sin ?

```
fn leer_archivo() -> Result<String, std::io::Error> {
    let mut archivo = File::open("hola.txt");
    match archivo {
        Ok(mut f) => {
            let mut contenido = String::new();
            match f.read_to_string(&mut contenido) {
                Ok(_) => Ok(contenido),
                Err(e) => Err(e),
            }
        }
        Err(e) => Err(e),
    }
}
```

Ejemplo con ?

```
fn leer_archivo() -> Result<String, std::io::Error> {
    let mut contenido = String::new();
    File::open("hola.txt")?.read_to_string(&mut contenido)?;
    Ok(contenido)
}
```

Patrones para manejar errores correctamente

match

```
match archivo {
    Ok(a) => println!("Abierto"),
    Err(e) => println!("Error: {}", e),
}
```

if let

```
if let Some(n) = x {
    println!("El número es {}", n);
}
```

while let

Ideal para iteradores:

```
let mut iter = vec![1,2,3].into_iter();
while let Some(v) = iter.next() {
    println!("{}", v);
}
```

:: Errores NO Recuperables

¿Qué es un error no recuperable?

Un error no recuperable es cuando la única opción segura es detener el programa inmediatamente para evitar corrupción de datos o comportamiento impredecible.

Ejemplos:

- Índice fuera de rango en un vector
- Dividir entre cero
- Usar unwrap() sobre None o Err
- Violaciones lógicas del código (invariantes imposibles)

Rust usa panic! para estos casos.

▶ panic!

Detener el programa de forma controlada

panic! es una macro que termina la ejecución y muestra un mensaje.

```
fn main() {
    panic!("Ocurrió un error inesperado");
}
```

Salida típica:

```
thread 'main' panicked at 'Ocurrió un error inesperado', src/main.rs:2:5
```

¿Cuándo ocurre un panic automáticamente?

1. Índices fuera de rango

```
let v = vec![1, 2, 3];
println!("{}", v[10]); // panic!
```

1. unwrap() sobre None

```
let x: Option<i32> = None;
x.unwrap(); // panic
```

1. expect() también provoca panic

```
let r: Result<i32, _> = "hola".parse();
r.expect("No se pudo convertir"); // panic con mensaje personalizado
```

Cuándo no usar unwrap() y expect()

Ambos métodos causan panic si:

- Option es None
- Result es Err

¿Por qué existen entonces?

Porque en prototipos, scripts rápidos y tests es conveniente usarlos.

Cuándo evitar unwrap/expect:

- En producción
- En código que maneja errores del usuario
- En operaciones de I/O, red, archivos
- Cuando el valor puede no existir o fallar

▶ Backtrace

seguir el origen del panic

Rust puede mostrar la ruta de llamadas que llevó al error.

Para activarlo:

```
RUST_BACKTRACE=1 cargo run
```

Ejemplo de salida:

```
thread 'main' panicked at 'fallo X', src/main.rs:12:9
stack backtrace:
   0: rust_begin_unwind
   1: core::panicking::panic_fmt
   2: my_project::funcion
   3: my_project::main
```

Esto es útil para depurar.

Modularidad

Cuando nuestro programa empieza a crecer nos enfrentamos con un codigo desorganizada todo en un solo archivo. Por esta razon, Rust nos ayuda con la organización de nuestro codigo, agrupando y separando distintas responsabilidades, funciones y características.

Rust usa un sistema de crates, packages, módulos y rutas para mantener el código limpio y reutilizable.

- **Packages** (Paquetes): Una función de carga que le permite construir, probar y compartir cajas
- **Crates** (Cajas): Un árbol de módulos que produce una biblioteca o ejecutable
- **Modules** (Módulos): Le permiten controlar la organización, el alcance y la privacidad de Caminos
- **Paths** (Rutas): una forma de nombrar un elemento, como una estructura, función o módulo

:: Packages and Crates

Un crate es la unidad mínima de compilación en Rust.

- Puede ser un ejecutable *"binary crate"*.
- O puede ser una librería reutilizable *"library crate"*.

Cuando usas `cargo build`, Cargo compila un crate.

▸ Tipos de crates

1. Binary crate
   - Contiene un main.rs.
   - Produce un ejecutable.

   Tiene un punto de entrada: la función `fn main()`

   Ejemplo: `cargo new hola_mundo`

   Esto crea un binary crate con src/main.rs.

   ```rust
   fn main() {
       println!("Hola, mundo!");
   }
   ```

   Se compila a un programa ejecutable.
2. Library crate
   - Contiene un lib.rs.
   - No tiene main().
   - Produce una librería que otros programas pueden usar.

   Ejemplo (lib.rs): `cargo new --lib mi_lib`

   Esto crea una library crate con src/lib.rs.

```rust
pub fn suma(a: i32, b: i32) -> i32 {
    a + b
}
```

Se puede usar desde otro crate con:

```rust
use mi_lib::suma;
```

Crates externos

Los crates externos son librerías publicadas en crates.io.

Se añaden en Cargo.toml:

```toml
[dependencies]
rand = "0.8"
```

Un Package es un conjunto de crates administrado con Cargo.

Cada package tiene un archivo Cargo.toml que describe:

- Dependencias.
- Nombre.
- Versión.
- Autores.

Ejemplo de estructura:

```
mi_proyecto/      <- package
 ├── Cargo.toml
 └── src/
      ├── main.rs <- binary crate
      └── lib.rs <- library crate
```

:: Module

- Los módulos sirven para organizar el código dentro de un crate.
- Agrupan funciones, structs, enums, etc.
- Permiten controlar la visibilidad (pub o privado).
- Evitan que todo quede en un solo archivo gigante.
- Se definen con `mod`.
- Pueden estar:
  - En el mismo archivo.
  - En archivos separados (mod nombre; busca nombre.rs o nombre/mod.rs).

Ejemplo en el mismo archivo:

```rust
mod matematicas {
```

```
    pub fn suma(a: i32, b: i32) -> i32 {
        a + b
    }

    fn resta(a: i32, b: i32) -> i32 {
        a - b
    }
}

fn main() {
    println!("Suma: 5 + 3 = {}", matematicas::suma(5, 3));
    // println!("Resta: {}", matematicas::resta(5, 3)); // ERROR: resta no es
    pública
}
```

Aquí `suma` es pública "`pub`" pero `resta` es privada.

El mismo ejemplo pero ahora usaremos `use` para acortar rutas:

```
mod matematicas {
    pub fn sumar(a: i32, b: i32) -> i32 {
        a + b
    }
}

use matematicas::sumar; // importamos

fn main() {
    println!("2 + 3 = {}", sumar(2, 3)); // más corto
}
```

También puedes importar todo:

```
use matematicas::*;
```

▸ Varios archivos

Cuando contamos con mas archivos como por ejemplo este:

```
src/
 ├── main.rs
 ├── utilidades.rs
```

1. `main.rs`

   ```
   mod utilidades; // le decimos que existe ese archivo

   fn main() {
       utilidades::saludo("Luis");
   ```

```
}
```

2. `utilidades.rs`

```rust
pub fn saludo(nombre: &str) {
    println!("Hola, {}!", nombre);
}
```

Ahora el código está separado.

▸ Varios archivos en carpetas

Supongamos esta estructura:

```
src/
 ├── main.rs
 ├── matematicas/
 │     ├── mod.rs
 │     ├── suma.rs
 │     └── resta.rs
```

1. `main.rs`:

```rust
mod matematicas;

fn main() {
    println!("2 + 3 = {}", matematicas::suma::sumar(2, 3));
}
```

2. `matematicas/mod.rs`:

```rust
pub mod suma;
pub mod resta;
```

3. `matematicas/suma.rs`:

```rust
pub fn sumar(a: i32, b: i32) -> i32 {
    a + b
}
```

4. `matematicas/resta.rs`:

```rust
pub fn restar(a: i32, b: i32) -> i32 {
    a - b
}
```

Asi conseguimos un módulo matematicas con submódulos suma y resta.

▸ Submódulos

Los módulos pueden tener submódulos:

```
mod libreria {
    pub mod libros {
        pub fn listar() {
            println!("Lista de libros...");
        }
    }
}

fn main() {
    libreria::libros::listar();
}
```

▸ Rutas (paths)

Las rutas indican cómo encontrar funciones, structs, enums, etc., dentro de los módulos.

Ruta absoluta: comienza desde la raíz del crate.

Ruta relativa: parte desde el módulo actual `self::` o `super::`.

Ejemplo:

```
mod animales {
    pub mod perro {
        pub fn ladrar() {
            println!("Guau!");
        }
    }
}

fn main() {
    // Ruta absoluta
    crate::animales::perro::ladrar();

    // Ruta relativa
    animales::perro::ladrar();
}
```

▸ Use

Es como un atajo o importación de ruta. En vez de escribir una ruta larga cada vez `crate::modulo::submodulo::funcion`, con `use` puedes traer ese nombre al scope actual y usarlo directamente.

En palabras simples : `use` ahorra escribir y hace el código más legible.

Ejemplo sin `use`:

```
mod util {
```

```
    pub fn saludar() {
        println!("Hola!");
    }
}

fn main() {
    crate::util::saludar(); // siempre tengo que escribir la ruta completa
}
```

Ejemplo con use:

```
mod util {
    pub fn saludar() {
        println!("Hola!");
    }
}

use crate::util::saludar;

fn main() {
    saludar(); // gracias al use ya puedo llamarla directo
}
```

▶ super::

- `super::` sirve para acceder al módulo padre desde un submódulo.
- Es como decir: *"sube un nivel en la jerarquía de módulos"*.

Supongamos esta estructura de archivos:

```
src/
 └── main.rs
```

Código:

```
mod padre {
    pub fn saludo_padre() {
        println!("👋 Hola desde el padre");
    }

    pub mod hijo {
        pub fn saludo_hijo() {
            println!("🙋 Hola desde el hijo");
        }

        pub fn usar_padre() {
            // Usamos super:: para subir al módulo padre
            super::saludo_padre();
```

```
        }
    }
}

fn main() {
    padre::hijo::saludo_hijo();   // Llamada directa al hijo
    padre::hijo::usar_padre();    // El hijo usa una función del padre
}
```

- padre es el módulo principal.
- hijo es un submódulo dentro de padre.
- Desde hijo, usamos `super::saludo_padre()` para subir al módulo padre.

En consola:

```
🙋 Hola desde el hijo
👋 Hola desde el padre
```

Ejemplo con varios niveles

Estructura:

```
src/
 └── main.rs
```

Código:

```
mod abuelo {
    pub fn saludo_abuelo() {
        println!("👴 Hola desde el abuelo");
    }

    pub mod padre {
        pub fn saludo_padre() {
            println!("👨 Hola desde el padre");
        }

        pub mod hijo {
            pub fn saludo_hijo() {
                println!("👶 Hola desde el hijo");
            }

            pub fn usar_padre_y_abuelo() {
                super::saludo_padre();       // sube al módulo padre
                super::super::saludo_abuelo(); // sube dos niveles
            }
        }
    }
```

```
}

fn main() {
    abuelo::padre::hijo::saludo_hijo();
    abuelo::padre::hijo::usar_padre_y_abuelo();
}
```

Salida:

```
👶 Hola desde el hijo

👴 Hola desde el padre

👴 Hola desde el abuelo
```

- `super::` : sube un nivel en la jerarquía de módulos.
- `super::super::` : sube dos niveles, y así sucesivamente.
- Se usa para organizar mejor el código y evitar rutas largas desde los submódulos.

▶ self::

- `self::` significa "este mismo módulo".

Se usa para:

- Referirse a elementos dentro del mismo módulo (funciones, structs, etc.).
- Dejar más explícito que estás trabajando con el módulo actual.
- Evitar confusión cuando importas nombres desde fuera.

Muchas veces no es obligatorio escribir `self::`, pero ayuda a dar claridad en proyectos grandes.

Ejemplo básico:

```
mod calculadora {
    // Función pública
    pub fn suma(a: i32, b: i32) -> i32 {
        a + b
    }

    // Función pública que llama a otra dentro del mismo módulo
    pub fn operaciones() {
        let resultado = self::suma(2, 3); // self:: -> este mismo módulo
        println!("Resultado: {}", resultado);
    }
}

fn main() {
    calculadora::operaciones();
```

```
}
```

Aquí `self::suma` indica que estamos llamando a suma dentro del módulo calculadora.

Podríamos haber escrito solo suma(2, 3), pero `self::` lo hace explícito.

Ejemplo con submódulos:

```
mod libreria {
    pub fn info() {
        println!("📚 Bienvenido a la librería!");
    }

    pub mod libros {
        pub fn listar() {
            println!("Lista de libros...");
        }

        pub fn mostrar() {
            // Llamamos a otra función del mismo módulo
            self::listar();
        }
    }
}

fn main() {
    libreria::libros::mostrar();
}
```

Salida:

```
Lista de libros...
```

Aquí `self::listar()` aclara que la función listar está dentro del mismo módulo libros.

Ejemplo con use self::

A veces `self::` se usa junto con `use` para importar del mismo módulo:

```
mod util {
    pub fn limpiar() {
        println!("🧹 Limpiando...");
    }

    pub fn ejecutar() {
        // Importamos funciones del mismo módulo
        use self::limpiar;

        limpiar();
    }
```

```
}

fn main() {
    util::ejecutar();
}
```

- `self::` : módulo actual.
- Se usa para:
  - Llamar funciones o usar elementos dentro del mismo módulo.
  - Hacer explícita la ruta (especialmente en proyectos grandes).
  - Evitar ambigüedades con otros módulos.
- Muchas veces se puede omitir, pero es útil como buena práctica de claridad.

▶ crate::

- `crate::` : significa la raíz del crate actual.
- Desde cualquier módulo dentro del crate, crate:: te permite navegar desde la raíz, sin importar dónde estés.

Es parecido a una ruta absoluta dentro de tu proyecto.

Mientras que `self::` y `super::` son rutas relativas.

Supongamos esta estructura de archivos:

```
src/
 └── main.rs
```

Código:

```
mod util {
    pub fn saludar() {
        println!("👋 Hola desde util!");
    }
}

fn main() {
    // Desde la raíz del crate (main.rs es la raíz)
    crate::util::saludar();
}
```

Aquí `crate::util::saludar()` es lo mismo que `util::saludar()`.

Pero si estuvieras en un submódulo más profundo, `crate::` siempre te lleva a la raíz.

Ejemplo con submódulos

Estructura:

```
src/
 └── main.rs
```

Codigo:

```rust
mod util {
    pub fn limpiar() {
        println!("🧹 Limpiando...");
    }

    pub mod avanzado {
        pub fn ejecutar() {
            // Desde aquí accedemos a la raíz del crate
            crate::util::limpiar();
        }
    }
}

fn main() {
    util::avanzado::ejecutar();
}
```

- Estamos en `util::avanzado`.
- Para llamar a `util::limpiar()` podríamos usar `super::super::util::limpiar()`, pero es más fácil y claro usar `crate::util::limpiar()`.

Ejemplo en librerías (lib.rs)

Estructura:

```
src/
 ├── lib.rs
 └── main.rs
```

lib.rs:

```rust
pub mod calculadora {
    pub fn sumar(a: i32, b: i32) -> i32 {
        a + b
    }
}
```

main.rs:

```rust
fn main() {
    let resultado = crate::calculadora::sumar(5, 3);
    println!("Resultado: {}", resultado);
}
```

Aquí `crate::` significa “este crate de librería es en la que estoy”.

▸ Control de privacidad

Es privado por defecto y solo accesible dentro del mismo módulo.

Con pub podemos usarlo en archivos accesible desde fuera.

- pub(crate): accesible solo dentro del crate.
- pub(super): accesible solo desde el módulo padre.

Ejemplo basico:

```
mod a {
    pub(crate) fn solo_crate() {}
    pub(super) fn solo_modulo_padre() {}
}
```

Estructura del proyecto

```
src/
 ├── main.rs
 ├── utilidades/
 │     ├── mod.rs
 │     ├── texto.rs
 │     └── numeros.rs
```

1. main.rs : Aquí arrancamos el programa y usamos los módulos.

```
mod utilidades; // importamos el módulo "utilidades"

use utilidades::texto;
use utilidades::numeros;

fn main() {
    println!("=== Proyecto con módulos ===");

    // Usamos el módulo de texto
    texto::saludo("Luis");

    // Usamos el módulo de números
    let suma = numeros::sumar(5, 3);
    println!("5 + 3 = {}", suma);

    // Ejemplo con pub(crate) → se puede usar dentro del mismo crate
    let cuadrado = numeros::cuadrado(4);
    println!("4^2 = {}", cuadrado);

    // Ejemplo con pub(super) → lo uso desde el padre (mod.rs)
    utilidades::texto::imprimir_desde_super();
}
```

2. `utilidades/mod.rs`
   - Archivo principal del módulo utilidades.
   - Aquí agrupamos los submódulos.
3. `utilidades/texto.rs` : Aquí jugamos con `pub` y `pub(super)`.

```rust
pub fn saludo(nombre: &str) {
    println!("Hola, {}!", nombre);
}

// Esta función es accesible solo para el módulo padre (mod.rs)
pub(super) fn secreto_super() {
    println!("Soy un secreto, solo visible desde utilidades::mod.rs");
}

// Esta función llama a la de arriba
pub fn imprimir_desde_super() {
    println!("Llamando a secreto_super desde texto:");
    super::texto::secreto_super(); // uso de pub(super)
}
```

4. `utilidades/numeros.rs` : Aquí jugamos con pub y pub(crate).

```rust
pub fn sumar(a: i32, b: i32) -> i32 {
    a + b
}

// pub(crate) → accesible solo dentro de este crate (no fuera del proyecto)
pub(crate) fn cuadrado(n: i32) -> i32 {
    n * n
}
```

Resultado cuando corremos con el siguiente comando: `cargo run`

Salida en consola:

```
=== Proyecto con módulos ===
Hola, Luis!
5 + 3 = 8
4^2 = 16
Llamando a secreto_super desde texto:
Soy un secreto, solo visible desde utilidades::mod.rs
```

- `pub` : cualquiera puede acceder (incluso otros proyectos).
- `pub(crate)` : solo accesible dentro del crate actual.
- `pub(super)` : accesible solo por el módulo padre inmediato.
- sin `pub` : solo dentro del mismo archivo.

ROBUSTEZ Y ABSTRACCIONES

:: Más Cargo y Crates.io (publicación de paquetes).

:: Testing en Rust (unit, integration, benches).

PARADIGMAS FUNCIONALES Y OO

:: Operator Overloading.

:: Programación Orientada a Objetos en Rust

:: Pattern Matching Avanzado

SISTEMAS Y CONCURRENCIA

:: Smart Pointers

:: Threads y Concurrencia.

:: Async/Await y Concurrencia Ligera.

:: I/O Avanzado

:: Features Avanzados: Macros, Unsafe Rust y FFI

Test

Un test es simplemente una función especial que verifica si una parte de tu programa funciona como debería.

En Rust es especialmente crítico porque el lenguaje busca seguridad y precisión, y los tests refuerzan esa filosofía.

Esto es clave porque Rust tiene un sistema de pruebas integrado en el lenguaje y en su herramienta oficial cargo, lo cual lo hace muy sólido para desarrollar software confiable.

Los tests en Rust son importantes porque:

- Garantizan confiabilidad evitando bugs antes de desplegar.
- Documentan comportamiento.
- Facilitan refactorizar, esto permite cambiar la implementación sin miedo, si los tests siguen pasando.
- Un bug corregido se puede convertir en un test, para que nunca vuelva a ocurrir.

Se marcan con el atributo `#[test]` y se ejecutan con el comando: `cargo test`

Ejemplo básico:

```
#[test]
fn suma_basica() {
    assert_eq!(2 + 2, 4);
}
```

Cuando corres cargo test, Rust compila tu proyecto y ejecuta automáticamente todas las funciones con test.

:: Herramientas de test en Rust

Rust incluye macros de aserción para comprobar condiciones:

1. `assert!(condición)`, falla si la condición es falsa.
2. `assert_eq!(a, b)`, falla si a != b.
3. `assert_ne!(a, b)`, falla si a == b.

Ejemplo:

```
#[test]
fn pruebas_de_aserciones() {
    let x = 5;
    assert!(x > 0);              // OK
    assert_eq!(x + 1, 6);        // OK
    assert_ne!(x * 2, 9);        // OK
}
```

:: Tipos de tests en Rust

Rust organiza las pruebas en tres categorías principales:

1. Unit tests (pruebas unitarias)
    - Verifican funciones pequeñas, en aislamiento.

- Se suelen escribir en el mismo archivo del código, dentro de un módulo especial mod `tests`.

```rust
pub fn sumar(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)] // solo se compila en modo test
mod tests {
    use super::*;

    #[test]
    fn test_sumar() {
        assert_eq!(sumar(2, 3), 5);
    }
}
```

2. Integration tests (pruebas de integración)
   - Se colocan en la carpeta `tests/`.
   - Usan el paquete como si fuera una librería externa, probando cómo interactúan las partes juntas.

   Ejemplo (tests/integration.rs):

```rust
use mi_crate; // se importa como un crate normal

#[test]
fn suma_integra() {
    assert_eq!(mi_crate::sumar(2, 2), 4);
}
```

3. Doc tests (tests en documentación)
   - Rust permite incluir tests directamente en la documentación.
   - Se escriben en comentarios `///` y se ejecutan con `cargo test`.

```rust
/// Suma dos números.
///
/// # Ejemplo
///
/// use mi_crate::sumar;
/// assert_eq!(sumar(2, 3), 5);
///
pub fn sumar(a: i32, b: i32) -> i32 {
    a + b
}
```

:: Tests avanzados

- Esperar un pánico

```rust
#[test]
#[should_panic]
fn prueba_panic() {
    panic!("esto debe fallar");
}
```

- Tests con `Result`

  En vez de `assert!`, puedes devolver `Result<(), String>`:

```rust
#[test]
fn prueba_resultado() -> Result<(), String> {
    if 2 + 2 == 4 {
        Ok(())
    } else {
        Err("Error matemático".to_string())
    }
}
```

- Tests paralelos

  Por defecto, `cargo test` corre tests en paralelo,esto es más rápido.

  Si quieres forzar secuencial: `cargo test -- --test-threads=1`

- Filtrar tests

  Puedes correr solo algunos tests: `cargo test suma`

Beneficios de hacer tests en Rust

1. Integración nativa : No necesitas frameworks externos, cargo test ya está integrado.
2. Velocidad : Los tests se compilan a binarios rápidos, no a intérpretes.
3. Paralelismo automático : Tests corren en múltiples hilos.
4. Documentación ejecutable : Los ejemplos en la documentación son tests reales.
5. Confianza : Rust + tests = software muy seguro y confiable.

Smart Pointers

Los Smart Pointers en Rust son fundamentales para entender cómo Rust maneja la memoria de manera segura y eficiente, sin un garbage collector.

En general, un pointer (puntero) es una variable que almacena una dirección de memoria.

En Rust, además de referencias normales `&T`, existen los Smart Pointers, que son estructuras que se comportan como punteros, pero con funcionalidad extra.

:: Tipos principales de Smart Pointers

Rust trae varios smart pointers en la librería estándar `std`.

1. Box<T> Heap Allocation
   - Guarda un valor en el heap en vez de en el stack.
   - Útil cuando no conoces el tamaño exacto en compilación o para evitar mover datos grandes en memoria.

```
fn main() {
    let x = Box::new(5);
    println!("x = {}", x);
}
```

   - Se comporta como un puntero a un i32.
   - Se libera automáticamente cuando sale de scope.

   Caso típico: estructuras recursivas (como listas enlazadas) donde el compilador necesita saber el tamaño.

```
enum Lista {
    Nodo(i32, Box<Lista>),
    Vacio,
}
```

2. Rc<T> Reference Counted
   - Conteo de referencias en tiempo de ejecución.
   - Permite múltiples dueños inmutables de un mismo valor.
   - Solo funciona en single-thread.

```
use std::rc::Rc;

fn main() {
    let a = Rc::new(5);
    let b = Rc::clone(&a);
    let c = Rc::clone(&a);

    println!("a={}, b={}, c={}", a, b, c);
    println!("Número de referencias: {}", Rc::strong_count(&a));
}
```

   Se libera cuando el contador llega a 0.

3. Arc<T> Atomic Reference Counted
   - Igual que `Rc<T>`, pero seguro para múltiples hilos (threads).
   - Usa operaciones atómicas, esto es un poco más lento que `Rc`.

```
use std::sync::Arc;
use std::thread;
```

```rust
fn main() {
    let valor = Arc::new(42);

    let mut handles = vec![];
    for _ in 0..5 {
        let v = Arc::clone(&valor);
        handles.push(thread::spawn(move || {
            println!("Valor = {}", v);
        }));
    }

    for h in handles {
        h.join().unwrap();
    }
}
```

4. RefCell<T> Borrowing dinámico
   - Permite reglas de préstamos en tiempo de ejecución en vez de compilación.
   - Ideal cuando necesitas mutabilidad interior.

```rust
use std::cell::RefCell;

fn main() {
    let x = RefCell::new(5);
    *x.borrow_mut() += 1;
    println!("{}", x.borrow()); // 6
}
```

   Si rompes las reglas (ej. dos mutables al mismo tiempo), `panic` en runtime.

5. Rc<RefCell<T>> Patrón muy usado

   Combina múltiples dueños (Rc) con mutabilidad interior (RefCell).

```rust
use std::rc::Rc;
use std::cell::RefCell;

fn main() {
    let dato = Rc::new(RefCell::new(5));

    let d1 = Rc::clone(&dato);
    let d2 = Rc::clone(&dato);

    *d1.borrow_mut() += 10;
    *d2.borrow_mut() += 20;

    println!("{}", dato.borrow()); // 35
```

```
}
```

Otros smart pointers útiles

- Cow<'a, T> : Copy-On-Write, evita copiar hasta que sea necesario.
- Mutex<T> : Exclusión mutua entre hilos.
- RwLock<T> : Permite múltiples lectores o un único escritor.

¿Por qué son importantes?

- Permiten estructuras de datos avanzadas (árboles, grafos, listas enlazadas).
- Controlan la memoria sin garbage collector.
- Son la base de concurrencia segura en Rust (Arc, Mutex, RwLock).
- Dan flexibilidad, eliges el smart pointer correcto según propietario, mutabilidad y concurrencia.

Asynchronous

Imagina por un instante que te pones a hervir agua para servirte una taza de café, sí, ese café que tanto te gusta, dime tú, te quedas esperando todo ese tiempo mirando a la tetera a que termine?
Lo más probable es que no, cierto? tú aprovechas ese tiempo para hacer otras cosas y ya cuando suene la tetera entonces retomas lo otro y te sirves el café.
Eso mismo es lo que hace la programación asíncrona, hace uso de los llamados threads o hilos de ejecución del procesador para que puedas ejecutar más de un proceso a la vez en tu aplicación.
Si no trabajas con programación asíncrona quiere decir entonces que tu aplicación siempre trabajará sobre un mismo hilo de ejecución y esto significa un problema, no solo de rendimiento por cuellos de botella (lentitud por convergencia de procesos), sino también de seguridad y disponibilidad, ya que es probable que tu aplicación deje de funcionar o se "crashee" debido a la demanda de procesos en ejecución al mismo tiempo.

:: Conceptos clave

1. `Future`
   - Representa un valor que estará disponible en el futuro.
   - Ejemplo: "el resultado de una petición HTTP".

```
use std::future::Future;

async fn foo() -> i32 {
    42
}

fn main() {
```

```
    let f = foo(); // f es un Future, aún no se ejecuta
    // Necesitamos un runtime para .await
}
```

2. `async` y `await`

- `async fn` : devuelve automáticamente un `Future`.
- `.await` : espera sin bloquear el hilo hasta que el `Future` esté listo.

```
async fn calcular() -> i32 {
    10 + 20
}

#[tokio::main] // runtime de Tokio
async fn main() {
    let resultado = calcular().await;
    println!("Resultado = {}", resultado);
}
```

Runtimes: `Tokio` y `async-std`

- `Tokio` : el más usado, ya que esta altamente optimizado.
- `async-std` : similar a Node.js, API simple.

El compilador Rust no trae runtime integrado, debes usar uno.

3. `Streams`

- Un `Future` produce un solo valor en el futuro.
- Un `Stream` produce múltiples valores en el tiempo.

Ejemplo:

```
use futures::stream::{self, StreamExt};

#[tokio::main]
async fn main() {
    let numeros = stream::iter(vec![1, 2, 3]);

    numeros.for_each(|x| async move {
        println!("Número: {}", x);
    }).await;
}
```

Ejemplo: Obtener datos de la API pública : `https://jsonplaceholder.typicode.com/posts/1`

Archivo `cargo.toml`:

```
[package]
name = "learningrust"
version = "0.1.0"
edition = "2024"

[dependencies]
tokio = { version = "1", features = ["full"] }
reqwest = { version = "0.11", features = ["json"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
```

Archivo `main.rs`

```
use reqwest;
use serde::Deserialize;

#[derive(Debug, Deserialize)]
struct Post {
    userId: i32,
    id: i32,
    title: String,
    body: String,
}

#[tokio::main] // ejecuta runtime Tokio
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Petición HTTP GET
    let resp = reqwest::get("https://jsonplaceholder.typicode.com/posts/1")
        .await?
        .json::<Post>()
        .await?;

    println!("Post recibido:\n{:#?}", resp);

    Ok(())
}
```

Aquí:

- `async fn main()` : el main es asincrónico gracias a tokio::main.
- `reqwest::get().await` : lanza la petición sin bloquear.
- `.json::<Post>().await?` : deserializa con serde el JSON en la struct Post.

Resultado:

```
Post recibido:
Post {
    userId: 1,
    id: 1,
    title: "sunt aut facere repellat provident occaecati excepturi optio
    reprehenderit",
    body: "quia et suscipit\nsuscipit..."
}
```

:: Streams con API

Si la API devuelve muchos datos (ej. lista de posts), puedes consumirlos como Stream:

```rust
use reqwest::Client;
use futures::StreamExt;
use serde::Deserialize;

#[derive(Debug, Deserialize)]
struct Post {
    userId: i32,
    id: i32,
    title: String,
    body: String,
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = Client::new();

    let resp = client
        .get("https://jsonplaceholder.typicode.com/posts")
        .send()
        .await?
        .json::<Vec<Post>>()
        .await?;

    futures::stream::iter(resp)
        .for_each(|post| async move {
            println!("Post {}: {}", post.id, post.title);
        })
        .await;

    Ok(())
}
```

Resumen

- Future = valor en el futuro (1 valor).
- async convierte funciones normales en funciones asincrónicas.
- await espera el resultado sin bloquear el hilo.
- Stream = múltiples valores asincrónicos.
- Necesitas un runtime como Tokio o async-std.
- Muy útil para redes, IO y APIs.

Conclusion

Bibliography

[1] Alex, “TITULO,” *JOUNA*, 2026.